VIETNAM NATIONAL UNIVERSITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications in Computer Vision

ASSIGNMENT E1_E2_E3

Instructor(s): Le Thanh Sach, Nguyen Quoc Minh

Author: Phung Bich Lan

Student ID: 2570438

HO CHI MINH, Oct, 2026

Source Code & Delivery package

A.1. Source repo: [CO5085_261_BTL_A1_PhungBichLan]

A.2. System environment

Parameter table	Detail configuration
Language & Library	Python 3.12.13 Pytorch Torchvision
Hardware	NVIDIA Google Colab
CUDA	13.2

A.3. Project directory structure

```
├── BTL_A1/
├──── source          // source code
├────── data          // data (train, validation, test)
├────── models        // model
├──── report          // report (.pdf)
├── BTL_A2/
├──── source          // source code
├────── data          // data (train, validation, test)
├────── models        // model
├──── report          // report (.pdf)
├── BTL_E1-E2-E3/
├──── data            // data (train, validation, test)
├────── E1
├──────── models      // model
├──────── .ipynb      // source code
├────── E2
├──────── models      // model
├──────── .ipynb      // source code
├────── E3
├──────── models      // model
├──────── .ipynb      // source code
├────── report        // report for E1, E2, E3
├── pages/            // .html of experiment
└── index.html        // main html of github page
```

A.4. Quick start commands

btl_e1_loadmodel.ipynb : to load best model and perform testing on test set

btl_e1.ipynb : to see the training progress

btl_e2_loadmodel.ipynb : to load best model and perform testing on test set

btl_e2.ipynb : to see the training progress

btl_e3_loadmodel.ipynb : to load best model and perform testing on test set

btl_e3.ipynb : to see the training progress

Project Execution / Timeline

Timeline	Task & Deliverables	Primary outputs
19/10/2026	E1-E3: GitHub pages	GitHub pages delivery for draft version
26/10/2026	A1.1: GitHub pages	GitHub pages deliver for Dataset + Protocol (data processing, training plan draft)
02/11/2026	E1, E2: source + report A2.1: GitHub pages	E1, E2: complete source and report A2.1: GitHub pages delivery for paper, dataset and metric
09/11/2026	E3: source + report A1.2: source + report	E3: complete source and report A1.2: complete source and report

Contents

Source Code & Delivery package 2
Project Execution / Timeline 4
List of Notations & Acronyms 8
Chapter 1. Introduction 9
1.1. Objectives 9
1.2. Scope of assignment 9
Chapter 2. General setup 10
2.1. Dataset & data preprocessing pipeline 10
2.1.1. Dataset selection & integration 10
2.1.2. Data preprocessing 11
2.1.3. Data loading & batching strategy 11
2.2. Common training setup 11
2.2.1. Cross Entropy Loss function 11
2.2.2. Optimizer and Scheduler 13
2.2.3. Hyperparameter 14
2.3. Common pipeline 15
2.3.1. Training pipeline 15
2.3.2. Evaluation pipeline 17
Chapter 3. Experiment E1 20
3.1. Specific requirement 20
3.2. Model architecture 20
3.2.1. Softmax classifier 20
3.2.2. MLP classifier 22
3.2.3. CNNs classifier 23
3.2.4. Model capacity summary 25
3.3. Experimental result and analysis 26
3.3.1. Overral performance comparison 26
3.3.2. Training and validation analysis with loss/accuracy curve 29

3.3.3. Classification performance analysis 30
3.3.4. Sample error analysis 31
3.4. Discussion & key takeaways 32
Chapter 4. Experiment E2 33
4.1. Specific requirement 33
4.2. Introduce attention mechanism 33
4.2.1. Objective 33
4.2.2. Scaled Dot-Product Attention 34
4.2.3. Multi-Head Self-Attention (MSA) overview 35
4.3. Model architecture 35
4.3.1. Multi-Head Self-Attention MSA 35
4.3.2. Transformer Encoder block 36
4.3.3. Image classifier design for experiment 37
4.4. Tokenization Strategies 39
4.4.1. Patch Tokenization 39
4.4.2. Pixel/Rows Tokenization 40
4.4.3. CNNs feature map Tokenization 41
4.5. Implementation details 41
4.5.1. Custom Tokenization 41
4.5.2. Custom projection & MSA 43
4.5.3. Custom Transformer Encoder 47
4.5.4. Custom Image Classifier 48
4.6. Experimental result and analysis 48
4.6.1. Custom MSA vs Pytorch Native MSA 48
4.6.2. Performance analysis base on different tokenization strategies 49
4.7. Discussion & key takeaways 56
Chapter 5. Experiment E3 57
5.1. Specific requirement 57
5.2. Sequential modeling pipeline 58

5.2.1. Recurrent neural networks 58
5.2.2. Image classifier design for experiment 62
5.3. Tokenization Strategies 62
5.4. Implementation sequential model 63
5.4.1. Long Short-Term Memory Networks (LSTM) 63
5.4.2. Gated Recurrent Unit Networks (GRU) 63
5.5. Experimental result and analysis 64
5.5.1. Comparison of different tokenization strategies 65
5.5.2. Comparison LSTM vs GRU 65
5.5.3. Comparison with E1 baseline (MLP/CNN) 70
5.6. Discussion & key takeaways 73
References 75

List of Notations & Acronyms

E1	Experiment 1
E2	Experiment 2
E3	Experiment 3
MLP	Multiple Layer Perceptrons
CNNs	Convilutional Neural Networks
MSA	Multi-head Self Attention
LSTM	Long Short Term Memory
GRU	Gated Recurrent Unit
CE	Cross Entropy
SGD	Stochastic Gradient Descent
TP	True Positive
FP	False Positive
TN	True Neagtive
FN	False Negative
ViT	Vision Transformer

Chapter 1. Introduction

1.1. Objectives

The primary objective of this assignment is to systematically explore, implement and evaluate three distinct deep learning architectural paradigms for “image classification” problem by using a shared dataset and a unified training setup (identical optimizer, scheduler and data processing pipeline).

After completing the assignment, following key expected as outcomes contain:

- Custom model implementation: understanding and implementation of baseline training loop for almost model; fundamental of MSA; sequence data transformation.
- Architecture trade-off analysis: insight into how different spatial and sequential feature extraction techniques (linear, MLP, CNNs, Self-Attention transformers and Recurrent RNNs) affect model capacity, parameter efficiency, training dynamics and classification accuracy.
- Experimental evaluation: well-founded assessment in analyzing model performance through evaluation metrics, loss-accuracy curves, confusion matrix and error analysis.

1.2. Scope of assignment

The assignment is structured into three main experiments and each focus on a specific architecture concept:

❖ E1 – baseline image classifier

Mainly focuses on establishing baseline performances using very simply concept without recurrent and attention model. It covers the end-to-end implementation of a custom training loop and the comparison evaluation of three core architecture: flatten Softmax linear classifier; Multi-Layer Perceptron (MLP); and Convolutional Neural Network (CNN).

❖ E2 – Multi-Head Self-Attention (MSA) and tokenization strategy

Mainly focuses on attention-based vision models. It invoslves building a Multi-Head Self-Attention (MSA) block from scratch (without reused package from Torchvision). Additionally explore and compare tokenization strategies.

❖ E3 – Sequential image modeling with Recurrent Neural Network (RNNs)

Mainly focuses on modeling 2D image spatial data as 1D sequential time-series inputs. By implement Long Short-Term Memory (LSTM) and Gated Recurrent Unit

(GRU) architectures using different sequence generation methods, make comparison with model baseline introduced in E1 (MLP/CNNs)

Chapter 2. General setup

2.1. Dataset & data preprocessing pipeline

2.1.1. Dataset selection & integration

In assignment, all experiments (E1, E2, E3) are conducted on the CIFAR-10 dataset which directly supported via Pytorch's native library (CIFAR10 — Torchvision main documentation)

Dataset overview: 60,000 RGB images (32x32 resolution) across 10 classes (separate to 50,000 training images and 10,000 test images)

Continues split dataset to one more subset called "validation set" from training set with ratio 0.8 for hyperparameter tuning, while the official test set (original 10,000 samples) is reserved strictly for final model evaluation.

Dataset	Number of samples	Input dimensions (C*H*W)
Train set	40,000	3x32x32
Validation set	10,000	3x32x32
Test set	10,000	3x32x32

The dataset exhibits a uniform class distribution, help to against class imbalance issues during model training phase.

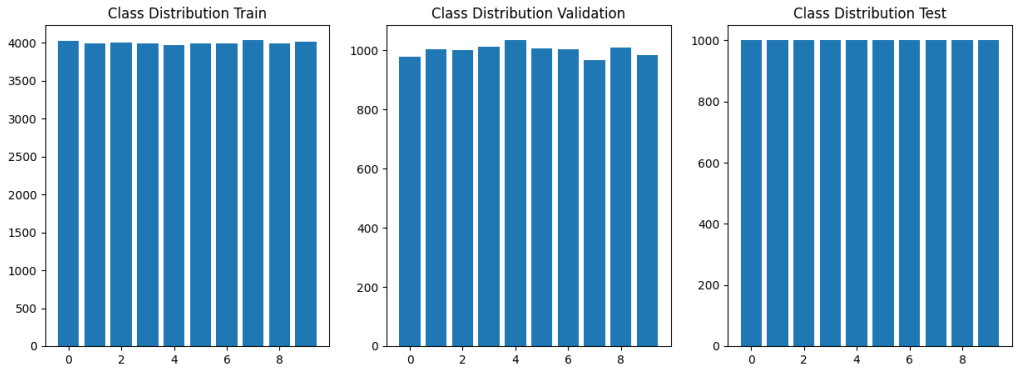


Figure 1. Class distribution for training/validation/testing set

This dataset is enough to show the effect of model architecture design for three key reason:

- Complexity sufficient: the 32x32 RGB image set provide enough real-world detail to evaluate how different models extract spatial feature via linear or non-linear layers
- Dataset size: 60,000 samples is not a large dataset but it is sufficient to evaluate the impact of model architecture design and observe training dynamics without requiring high-end computing resources
- Open source: simply download from Pytorch as baseline

2.1.2. Data preprocessing

To ensure strict and fair evaluation across E1, E2 and E3, all models share the same data pipeline using torchvision.transforms:

- Normalization: simple standardized with min-max scaling to [-1, 1] range
- Data augmentation (training set only): no apply to strictly evaluate model performance and architect capacity in their design, ensuring that accuracy effect come from model design rather than data generation
- Tensor conversion: applied via transforms.ToTensor()

2.1.3. Data loading & batching strategy

All dataset are wrapped in Pytorch `torchvision.datasets` and streamed using `torch.utils.data.DataLoader` to optimize memory consumption and GPU utilization during training loop:

- Batch size: number of training samples processed before the model update its parameter. It strongly impact to computational efficiency, gradient estimation, memory usage and model generalization. A small batch size produces noiser gradient estimate due to calculate from fewer samples. But sometime, the noise help the optimizer escape local minima or poor solution, but it also make the training unstable. Larger batch size provide smoother update.
- Shuffling: enabled for training set at each epoch to break sample correlation; disabled for validation and test sets to maintain evaluation consistency
- Multi-processing: config with `num_workers=8`, `pin_memory=True` to accelerate data transfer from CPU memory to GPU VRAM

2.2. Common training setup

2.2.1. Cross Entropy Loss function

In classification problems, model predicts the probability of each class for any given input. Due to each data point belongs to only one class, the expected output should be

probability 1 for true class, 0 for others. Then, cross-entropy loss is a way to measure how close a model's predictions are to the expected result.

Apply cross entropy for multiclass classification, it introduces multiclass cross entropy loss: $CE = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{C}\left(y_{i,j}\log\left(p_{i,j}\right)\right)$, where:

- N: number of samples
- C: number of classes
- $y_{i,j}$: 1 if class j is correct for sample i, 0 otherwise
- $y_{i,j}$: model-predicted probability of sample i being in class j

Interpretability for multiclass classification using cross entropy loss function:

- Only predicted same as true label contributes towards the loss as for other labels being zero does not affect to loss computation.
- Heavily penalize the wrong class predict by model then the cross entropy result becomes larger to indicate the high loss.

Module `torch.nn` provides essential loss function `nn.CrossEntropyLoss` for multi-class classification task which combines `LogSoftmax` on an input, followed by `NLLLoss` (negative log likelihood loss) [2]

To prove this, manually calculate `nn.LogSoftmax` and apply `nn.NLLLoss` on the output then compare with result output of `nn.CrossEntropyLoss` by following statement:

```
logits = torch.tensor([[2.0, 1.0, 0.1]])                # batch size = 1, classes = 3
label = torch.tensor([0])                               # true class = 0

ce_lossfunction = nn.CrossEntropyLoss()                 # LogSoftmax + NLLLoss
ce_loss = ce_lf(logits, label)                          # loss calculate

logit_softmax = nn.LogSoftmax(dim=1)
nllloss = nn.NLLLoss()

logit_prob = logit_softmax(logits)                      # apply softmax to logits
logit_softmax_loss = nllloss(logit_prob, labels)        # apply NLLLoss

logit_loss = nllloss(logits, label)                     # apply only NLLLoss
```

```
…
Execute
…
```

```
using nn.CrossEntropyLoss : 0.417030
manually (LogSoftmax + nn.NLLLoss) : 0.417030
manually nn.NLLLoss : -2.000000
nn.CrossEntropyLoss is equivalent to manually (LogSoftmax + NLLLoss)
```

From result, we can directly using `nn.CrossEntropyLoss` apply to logit output from model without softmax non-linear activation in output layer.

2.2.2. Optimizer and Scheduler

Design the `modelOptimizer` class to manage the optimization process combine with a learning rate scheduler which play a crucial role in determining model convergence, stability and final performance.

Introducing SGD (Stochastic Gradient Descent) as optimizer algorithm that updates model patameters using computed gradient of the loss function on mini-batch of data. Develop from traditional Gradient Descent, SGD is calculated for each training sample (a subset of training samples) rather than the entire dataset. `momentum` controls the influence of previous gradients on parameter updates, while `weight_decay` helps prevent excessively large weights.

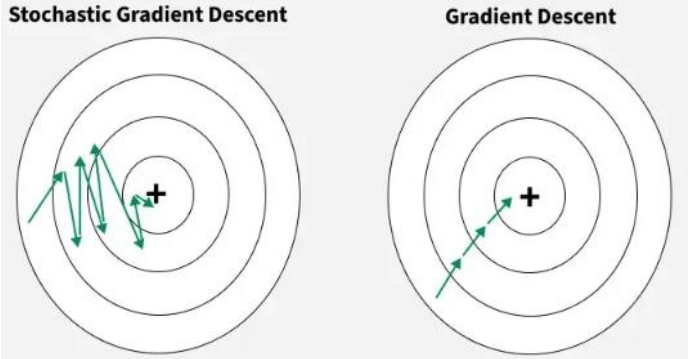


Figure 2. Stochastic Gradient Descent (SGD) and Gradient Descent (GD)

`StepLR` scheduler introduce to decay learning rate (`lr`) of each parameter group by `gamma` every `step_size` epoch. Example for scheduler StepLR with `step_size=30`, `gamma=0.1` from initialize learning rate `lr=0.05`:

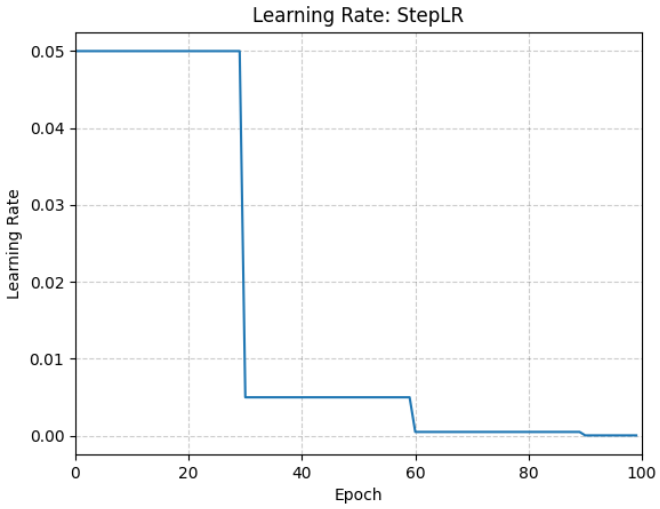


Figure 3. StepLE scheduler (lr=0.05, step_size=30, gamma=0.1)

Class definition with combine optimizer SGD and scheduler StepLE as following implementation:

```
class modelOptimizer():
    def __init__(self, model, lr):
        self.model = model                                  # model apply
        self.lr = lr                                        # learning rate
        self.optimizer =                                    # optimizer SGD
torch.optim.SGD(self.model.parameters(), lr=self.lr,
momentum=mm, weight_decay=weight_decay)
        self.scheduler =                                    # StepLR scheduler
torch.optim.lr_scheduler.StepLR(self.optimizer,
step_size=step_size, gamma=gamma)

    def zero_grad(self):                                    # reset gradients of all
        self.optimizer.zero_grad(set_to_none=True)          parameter to 0

    def step(self):                                         # update model weights base
        self.optimizer.step()                               on computed gradients

    def scheduler_step(self):                               # adjust learning rate by
        self.scheduler.step()                               schedule
```

2.2.3. Hyperparameter

Hyperparameter	Value	Apply	Explain
batch_size	32	E1, E2, E3	Total training sample = 4000, batch size 32 => 125 batchs in epoch Batch size = 32 is balance with computation efficiency with time and memory
lr	0.01	E1, E2, E3	Choose a relatively large learning rate if training from scratch
mm	0.9	E1, E2, E3	Accelerates convergence by incorporating information from previous gradients and helps reduce oscillations during optimization
weight_decay	0.0005	E1, E2, E3	Applies L2-style regularization to discourage excessively large weights and help reduce overfitting
num_epochs	50 70	E1, E3 E2	50 epoch for E1, E3 70 epoch for E2 allowing the more complex Transformer models additional training iterations
step_size	10 30	E1, E3 E2	Step size = 10 for E1 (to avoid overfitting that model remember training data) Step size = 30 for E2 (allowing the ViT models more epochs to learn before the first learning rate reduction)
gamma	0.1	E1, E2, E3	Multiplies the learning rate by 0.1 whenever the scheduler reaches the specified step size
num_worker	8	E1, E2, E3	Uses 8 subprocesses for data loading to enable parallel data preparation and reduce data-loading overhead
patch_size	4*4	E1, E2, E3	Apply a 4*4 patch size to input image for convert from raw pixel to token representation
D_ff	256	E1, E2, E3	Hidden dimension of FFN
msa_head	8	E1, E2, E3	Number of attention heads in Multi-Head Self-Attention

msa_dim	128	E1, E2, E3	Embedding dimension of each token, each head operates on a dimension of msa_dim/msa_head
num_encoder	3	E1, E2, E3	Transformer encoder block sequentially

2.3. Common pipeline

2.3.1. Training pipeline

Image classification pipeline defines the complete workflow of data processing, model training, validation, and final testing. It begins with loading and preprocessing the dataset, follow by feeding training batches into the model to generate predictions and calculate the loss. The model parameters are then updated through backpropagation with apply an optimization algorithm and learning rate scheduler also consider. After each epoch, the model is evaluated on the validation set to monitor performance and select the best-performing checkpoint. Finally, the selected best model is evaluated on the independent test set to measure its generalization performance.

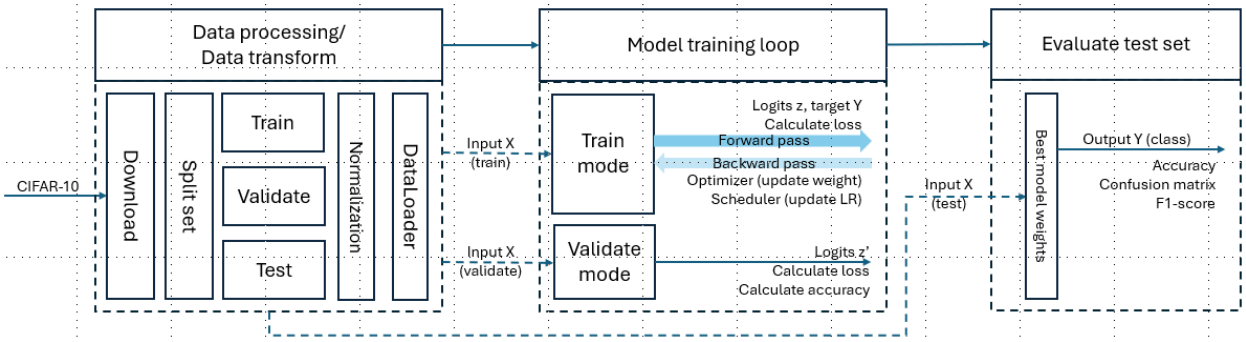


Figure 4. Image classification pipeline

The implementation covers the complete image classification workflow, from data preparation and model training to validation, best-model selection, and final testing. The common implementation consists of the following components, which are shared across all models. The specific models are described separately.

train_dataset = torchvision.datasets.CIFAR10(train=True, download=True) test_dataset = torchvision.datasets.CIFAR10(train=False, download=True)	[Download] Download dataset CIFAR-10 from Torchvision train=True : training set train=False : validate set
train_splitset, val_splitset = random_split(train_dataset, [train_size, val_size], generator=torch.Generator().manual_seed(3)) train_size/val_size = 80/20	[Split set] Split train_dataset to train_splitset and val_splitset, manual seed a specific value to ensure reproducibility by making random operations consistent across multiple runs
transform_cifar = transforms.Compose([	[Normalization]

transforms.ToTensor(), transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))]) train_splitset.dataset.transform = transform_cifar val_splitset.dataset.transform = transform_cifar test_dataset.transform = transform_cifar	Normalization input pixel value to range [-1,1] Does not apply any argumentation to ensure a fair comparison of the models under the same data conditions and allowing performance differences to be attributed primarily to model architecture
train_loader = torch.utils.data.DataLoader(train_splitset, batch_size=batch_size, shuffle=True, num_workers=8, pin_memory=True) val_loader = torch.utils.data.DataLoader(val_splitset, batch_size=batch_size, shuffle=False, num_workers=num_workers, pin_memory=True) test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=num_workers, pin_memory=True)	[DataLoader] Use PyTorch DataLoader to load the training, validation, and test dataset in mini-batch. Enable shuffle the training data at each epoch to improve training efficiency, while keeping validation and test data in a fixed order for consistent evaluation. Use multiple worker processes (num_workers) to load data in parallel. Enable pin_memory to potentially accelerate data transfer from CPU to GPU.
nn.CrossEntropyLoss() : Using CrossEntropy loss function for multiple class classification torch.optim.SGD : design an optimizer with learning rate, momentum and weigh_decay from Hyperparameter torch.optim.lr_scheduler.StepLR : design StepLR scheduler with step_size and gamma from Hyperparameter	[Common training setup] Ensure all common training setup are ready
Specific model architecture design (model) model_softmaxcls = SoftmaxClassifierDefinition() model_mlp = MulLayerPer() model_CNNs = DeepCNN()	[Model creation] This part should be specific for each model
[Train mode] for each epoch: model.train() (1) for each mini-batch: logits_z = model(input_x) (2) model_optimizer.zero_grad (3) loss backward (4) model_optimizer.step (5)	[Model training loop] (1) Enable training mode (2) logits_z from model (3) reset gradients of all parameter to 0 (4) calculate gradient (5) update weights

calculate loss/accuracy of training set model_optimizer.scheduler_step (6) update global loss min, global accuracy max [Evaluate mode] for each epoch: model.eval() (7) with torch.no_grad(), for each mini-batch (8) logits_z = model(input_x) (9) calculate loss/accuracy of validate set update global loss min, global accuracy max [Save checkpoint] for each epoch: save last checkpoint save best model check point (10) [Save time] Time of training loop	(6) update LR (7) Enable evaluate mode (8) Disabling gradient calculation (9) logits_z from model (10) save best model which testing on evaluate test provide the best accuracy
best_model.load() best_model.eval() (1) with torch.no_grad(), for each mini-batch (2) logits_z = best_model(input_x) (3) calculate loss/accuracy of test set save prediction results	[Evaluate test set] (1) Enable evaluate mode (2) Disabling gradient calculation (3) logits_z from best_model
To evaluate model generalization: + Loss/Accuracy – epoch (train set) + Loss/Accuracy – epoch (validate set) + Loss/Accuracy on test set Confusion matrix	[Evaluation test set]

2.3.2. Evaluation pipeline

To evaluate the performance and generalization ability of the image classification model, a consistent evaluation pipeline is applied to all 3 models.

❖ Loss and accuracy

Training set: monitor loss/accuracy value after each epoch training to evaluate the model's learning progress. If loss value does not decrease significant, or the accuracy remains low, the model may be underfitting, the learning rate may be inappropriate, or the model may require further optimization. However, very high training accuracy or very low loss does not necessarily indicate the good generalization, model may be overfitting on the data training. Consequently, the model is highly confident when using its training data, but it performs poorly when evaluated on other datasets.

Validation set: evaluate after each training epoch using a separate validation dataset that is not used to update the model parameters. The purpose is to measure how well the model generalizes to unseen data during training. A decreasing validation loss and increasing validation accuracy indicate improved generalization. If training loss

continues to decrease while validation loss increases, the model may be overfitting the training data. Comparing training and validation metrics helps identify overfitting or underfitting and select an appropriate model checkpoint.

Test set: evaluated on test set after training with best model (which release the best accuracy on validation set) to measure the final classification performance on unseen data.

So, the model visualizes a goof generalization when both training and validation loss low and close to each other, while training and validation accuracy are both high and similar. This indicates that the model learn effectively and generalizes well to unseen data. Finally, perform evaluate on test set expects the low loss and high accuracy.

❖ Confusion Matrix and Precision, Recall, and F1-score

It uses to analyze classification results for each class, showing correct predictions and misclassifications. It helps identify frequently confused classes and differences in classification performance across classes. These observations may indicate potential class imbalance, insufficient training samples, or similarities between classes.

From matrix results, further analysis and appropriate adjustments, such as collecting more representative data, applying class weighting, or using data augmentation, can be performed to improve the classification performance.

With predicted class and known correct class, confusion matrix introduces 4 important evaluate metrics:

- TN (True Negative): the number of samples that are correctly predicted as negative (predicted class = Y''), meaning the actual class is negative (correct class = Y'')
- TP (True Positive): The number of samples that are correctly predicted as positive (predicted class = Y'), meaning the actual class is positive (correct class = Y')
- FN (False Negative): The number of samples that are incorrectly predicted as negative (predicted class =Y''), meaning the actual class is positive (correct class = Y')
- FP (False Positive): The number of samples that are incorrectly predicted as positive (predicted class = Y'), meaning the actual class is negative (correct class = Y'')

Confusion Matrix		Predicted Class		
		1	2	3
Actual Class	1	TN	FP	TN
	2	FN	TP	FN
	3	TN	FP	TN

Figure 5. Confusion matrix of multiple-class classification (class = 3)

Beside these confusion matrix metrics, Precision, Recall and F1-score provide a more detailed evaluation of class-wise performance which calculate from TN, TP, FN, FP.

- Precision: Measures the percentage of correctly predicted positive samples out of the total number of samples predicted as positive. A higher precision indicates a lower number of False Positives (FP)

$$Precision = \frac{TP}{TP + FP}$$

- Recall: Measures the percentage of correctly predicted positive samples out of the total number of positive samples. A higher recall indicates fewer False Negatives (FN)

$$Recall = \frac{TP}{TP + FN}$$

- F1-score: The harmonic mean of precision and recall, provide a balanced measure of both metrics. A higher F1-score indicates a better balance between precision and recall.

$$F1 - score = \frac{2}{\frac{1}{Percision} + \frac{1}{Recall}}$$

+ High Precision, Low Recall: the model makes a few positive errors (FP), but it misses many actual positive samples (FN). Its positive predictions seems reliable, but it fails to identify many positive samples (FN)

+ Low Percision, High Recall: the model identifies most actual positive samples (TP) by visualize fewer FN, but it produces many false positive samples (FP). The model capture most potive samples, however, its positive predictions are less reliable

+ High Precision, High Recall (near 1): the model identifies many actual positive samples (few FN) and correctly identifies most positive samples while making a few false positive error (few FP). This result indicates the strong classification performance
+ Low Percision, Low Recall (near 0): the model produces many false positive and false negative errors, indicating poor performance in identifying the positive class.

In conclusion, the model demonstrates strong performance when it achieves a good balance between Precision and Recall, correctly identifying most positive samples while making few false positive and false negative errors.

Chapter 3. Experiment E1

3.1. Specific requirement

(1) Implement three image classification models:
- Softmax Classifier: flatten each input image into a one-dimensional vector, apply a linear transformation to obtain class logits and use softmax to convert the logits into class probabilities.
- MLP Classifier: flatten each input image into a one-dimensional vector as a multiple fully connected layer contains hidden layer and nonlinear activation functions, end by an output layer of classification.
- CNNs Classifier: apply convolutional and pooling layers to extract spatial features from the input images (optional batch normalization) and pass the extracted features to a classification head to predict the output classes.

(2) Training and evaluation: implement a custom training loop for model training and evaluation.

(3) Model comparison and error analysis: compare three models in terms of classification accuracy and number of trainable parameters; show loss (train/validation) and accuracy curves; generate confusion matrix and visualize sample which wrong predict by model.

3.2. Model architecture

3.2.1. Softmax classifier

From experiment E1, Softmax introduces as a baseline model which quite simple on design and less layer in model. Convert the input image (CIFAR-10) from a PIL image format to a tensor, then flatten the tensor into a one-dimensional feature vector before feeding it into the Softmax layer. After flatten, model use all input features to predict the scores for each class which called fully connected layer introduce as Softmax layer. The highest

predicy score belongs to corresponding class. The higher the score, the more confident the model is that the predicted class is the correct one.

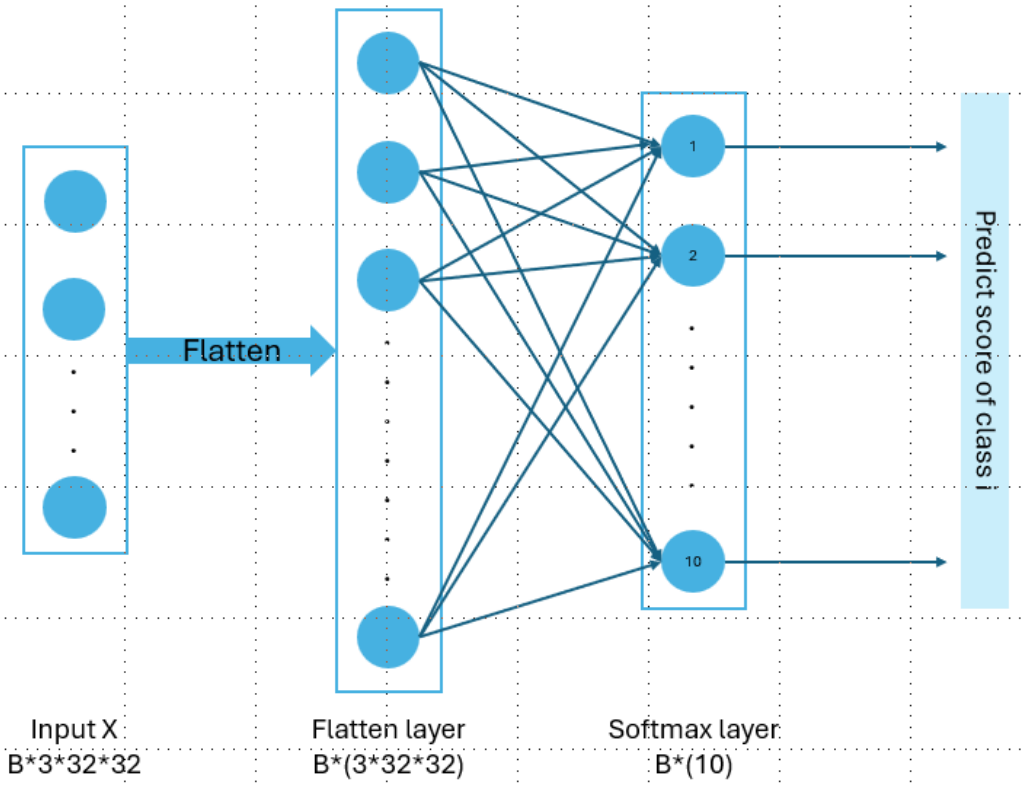


Figure 6. Softmax classifier architecture for image classification (B - batch_size)

Based on the architecture, implement a custom Softmax Classifier consisting of only one flatten layer and one linear output layer to predict class scores directly from the input image. The model does not explicitly apply Softmax to its output because `nn.CrossEntropyLoss` combines `nn.LogSoftmax` and `nn.NLLLoss` internally. Therefore, the model directly outputs raw logits from the linear layer.

```
class SoftmaxClassifierDefinition(nn.Module):
    def __init__(self, input_dim=3072, num_classes=10):
        super(SoftmaxClassifierDefinition, self).__init__()
        self.flatten = nn.Flatten()                          (1)
        self.linear = nn.Linear(input_dim, num_classes)      (2)

    def forward(self, x):                                    (3)
        x = self.flatten(x)
        x_linear = self.linear(x)
        return x_linear
```

```
dim=3072 (=32*32*3)

(1) flatten layer (no
weight), reshape only
(2) linear layer
(weight modify)
(3) forward pass
```

Total parameter can calculate by (with B = 32):

- Flatten layer: no parameter, reshape only
- Softmax layer: `(3*32*32)*10 + 1*10 (bias) = 30730`

⇨ Total parameter: `30730` (trainable = 30730, non-trainable = 0)

3.2.2. MLP classifier

Different from baseline Softmax model, Multiple Perceptron Layer model introduces hidden layers with nonlinear activation function, it allows the model to learn more complex patterns and relationships in the input data. Covert to tensor data and flatten it to one-dimensional vector is necessary same as baseline model. After flatten, the input features pass through multiple fully connected hidden layers with nonlinear activation function, followed by the output layer to predict class scores.

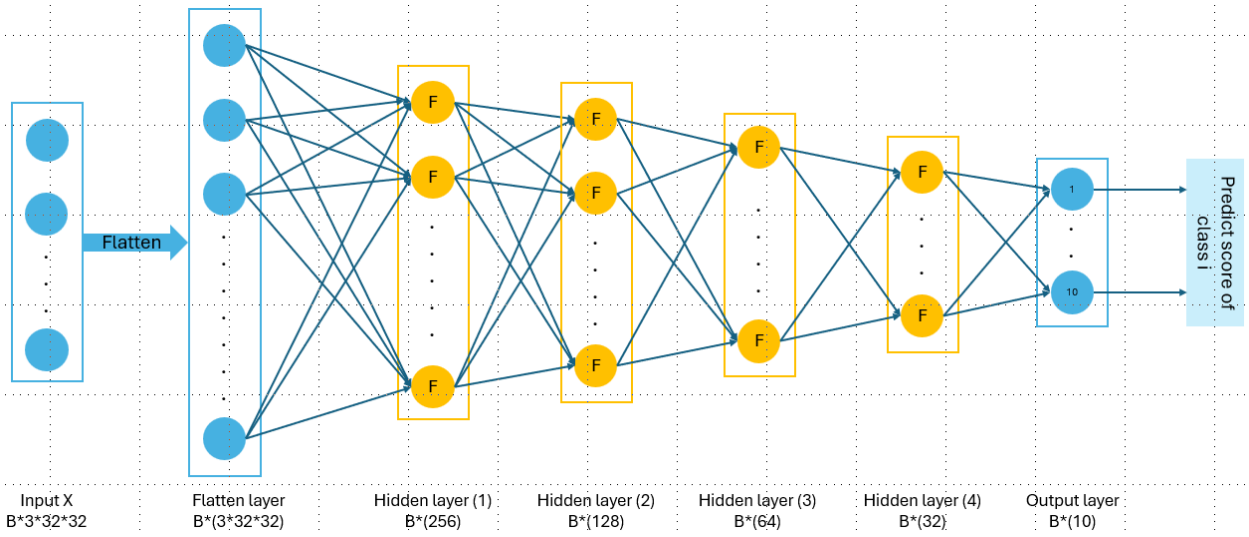


Figure 7. Multiple Layer Perceptron classifier architecture for image classification (B - batch_size)

Design a custom MLP class, also it does not explicitly apply Softmax to its output.

```
class MulLayerPer(nn.Module):
    def __init__(self,
                 input_dim=3072,
                 num_perceptron_layer=[256, 128, 64, 32],
                 num_cls = 10):
        super(MulLayerPer, self).__init__()
        self.flatten = nn.Flatten()                                    (1)
        self.layers = nn.ModuleList()
        in_features = input_dim
        for out_features in num_perceptron_layer:
            self.layers.append(nn.Linear(in_features, out_features))   (2)
            self.layers.append(nn.ReLU())                              (3)
            in_features = out_features                                 (4)
        self.layers.append(nn.Linear(in_features, num_cls))            (5)

    def forward(self, x):                                              (6)
        return self._forward_impl(x)

    def _forward_impl(self, x):                                        (6)
```

```
dim=3072 (=32*32*3)

(1) flatten layer
(no weight), reshape
only
(2) linear layer
(weight modify)
(3) Relu activation
function
(4) output of
previous layer be
input of next layer
(5) output linear
layer
(6) forward pass
through all hidder
layer
```

```
        out = self.flatten(x)
        for layer in self.layers:
            out = layer(out)
        return out
```

Total parameter can calculate by (with B = 32):

- Flatten layer: no parameter, reshape only
- Hidden layer (1): `(3*32*32)*256 + 1*256 (bias) = 786688`
- Hidden layer (2): `(256)*128 + 1*128 (bias) = 32896`
- Hidden layer (3): `(128)*64 + 1*64 (bias) = 8256`
- Hidden layer (4): `(64)*32 + 1*32 (bias) = 2080`
- Output layer: `(32)*10 + 1*10 (bias) = 330`

⇨ Total parameter: `830250` (trainable = 830250, non-trainable = 0)

3.2.3. CNNs classifier

Same hidden layer concept but unlike MLP model, which uses fully connected to learn features, CNNs uses convolutional blocks with nonlinear activation functions and max pooling to extract spatial features from images. Global Average Pooling then aggregates the feature maps before the final fully connected layer predicts class scores.

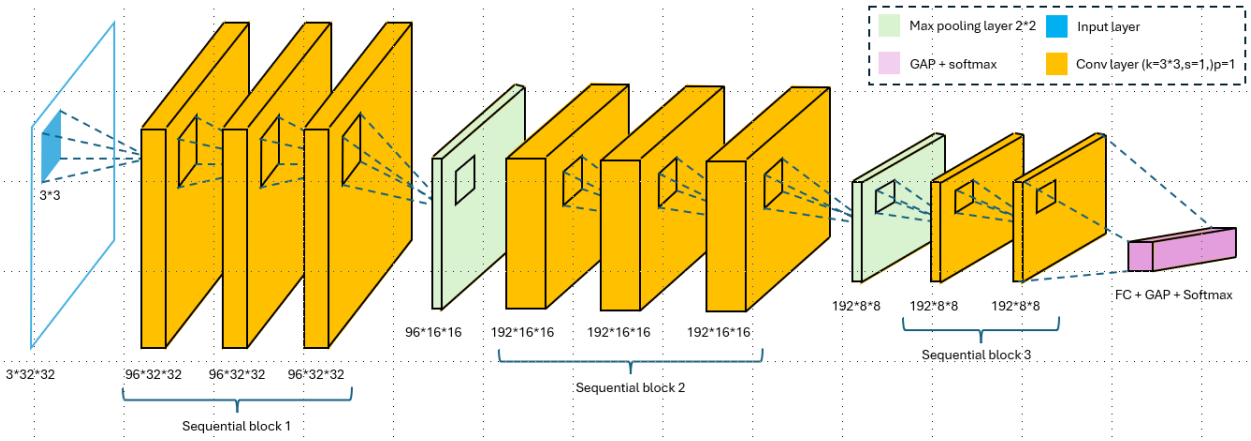


Figure 8. Convolutional Neural Network classifier architecture for image classification (B - batch_size)

Design sequential CNNs blocks, each consisting of convolutional layers with batch normalization during training to stabilize learning and potentially accelerate convergence; then apply non-linear activation functions to extract image features.

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size, padding, stride):
        super().__init__()

        self.conv = nn.Conv2d(in_ch,                                    (1)
                              out_ch,
                              kernel_size,
```

(1)(2) Convolution layer + BatchNorm
(3) Inside a ConvBlock
(4) Apply convolution with

stride, padding) self.bn = nn.BatchNorm2d(out_ch) (2) def forward(self, x): (3) x = self.conv(x) (4) x = self.bn(x) (5) x = F.relu(x) (6) return x	kernel_size, stride, padding (5) Apply batch normalization (6) Apply non-linear activation

class DeepCNN(nn.Module):
def __init__(self, visualize_featuremap=False):
super().__init__()
self.visualize_featuremap = visualize_featuremap
 	
self.block1 = nn.Sequential((1)
ConvBlock(3, 32, 3, 1, 1),
ConvBlock(32, 96, 3, 1, 1),
ConvBlock(96, 96, 3, 1, 1),
ConvBlock(96, 96, 3, 1, 1)
)
 	
self.block2 = nn.Sequential((2)
ConvBlock(96, 192, 3, 1, 1),
ConvBlock(192, 192, 3, 1,1),
ConvBlock(192, 192, 3, 1, 1)
)
 	
self.block3 = nn.Sequential((3)
ConvBlock(192, 192, 3, 1, 1),
ConvBlock(192, 192, 3, 1, 1)
)
 	
self.pool = nn.MaxPool2d(2, 2) (4)
self.gap = nn.AdaptiveAvgPool2d((1,1)) (5)
self.fc = nn.Linear(192, 10)
 	
def forward(self, x):
 	
features = {}
x = self.pool(self.block1(x)) (4)
features["block1"] = x
 	
x = self.pool(self.block2(x)) (4)
features["block2"] = x
 	
x = self.block3(x)
features["block3"] = x
 	
x = self.gap(x) (5)
 	
x = torch.flatten(x, 1) (6)
 	
out = self.fc(x) (7)
 	
if self.visualize_featuremap:
	(1) A sequential block with 4 convolution block (conv + batchnorm + relu)
	(2) A sequential block with 3 convolution block (conv + batchnorm + relu)
	(3) A sequential block with 2 convolution block (conv + batchnorm + relu)
	(4) Max pooling 2*2
	(5) GAP 1*1
	(6) Flatten
	(7) output linear layer

```
            return out, features
        return out
```

Total parameter can calculate by (with B = 32):

- Flatten layer: no parameter, reshape only
- Sequential block (1):
  + (0) Conv block: (3*3*3)*96 + 1*96 (bias) = 2688
  + (0) Batch Norm: 1*96 + 1*96 (bias) = 192
  + (1) Conv block: (3*3*96)*96 + 1*96 (bias) = 83040
  + (1) Batch Norm: 1*96 + 1*96 (bias) = 192
  + (2) Conv block: (3*3*96)*96 + 1*96 (bias) = 83040
  + (2) Batch Norm: 1*96 + 1*96 (bias) = 192
  => Total parameter of sequential block (1) = 169344
- Sequential block (2):
  + (0) Conv block: (3*3*96)*192 + 1*192 (bias) = 166080
  + (0) Batch Norm: 1*192 + 1*192 (bias) = 384
  + (1) Conv block: (3*3*192)*192 + 1*192 (bias) = 331968
  + (1) Batch Norm: 1*192 + 1*192 (bias) = 384
  + (2) Conv block: (3*3*192)*192 + 1*192 (bias) = 331968
  + (2) Batch Norm: 1*192 + 1*192 (bias) = 384
  => Total parameter of sequential block (2) = 831168
- Sequential block (3):
  + (0) Conv block: (3*3*192)*192 + 1*192 (bias) = 331968
  + (0) Batch Norm: 1*192 + 1*192 (bias) = 384
  + (1) Conv block: (3*3*192)*192 + 1*192 (bias) = 331968
  + (1) Batch Norm: 1*192 + 1*192 (bias) = 384
  => Total parameter of sequential block (2) = 664704
- FC layer + GAP: flatten, reshape and calculate average value of each channel only
- Output layer: (192)*10 + 1*10 (bias) = 1930

⇨ Total parameter: 1667146 (trainable = 1667146, non-trainable = 0)

3.2.4. Model capacity summary

Model capacity of three models is evaluated by computing their numbers of trainable parameter. Softmax classifier introduces a linear regression that maps input feature directly to class probabilities, it has low model capacity without hidden layer. MLP classifier consists of hidden layers with more than one fully connected layer applied activation function to learn complex patterns and non-linear relationships in input. CNNs uses convolutional layers to extract spatial features make network becomes deeper and more filter, the number of parameters and computational cost can increase significantly, depending on the

architecture. However, convolutional weight sharing generally makes CNNs more parameter-efficient than fully connected networks (MLP) for intput data such as images.

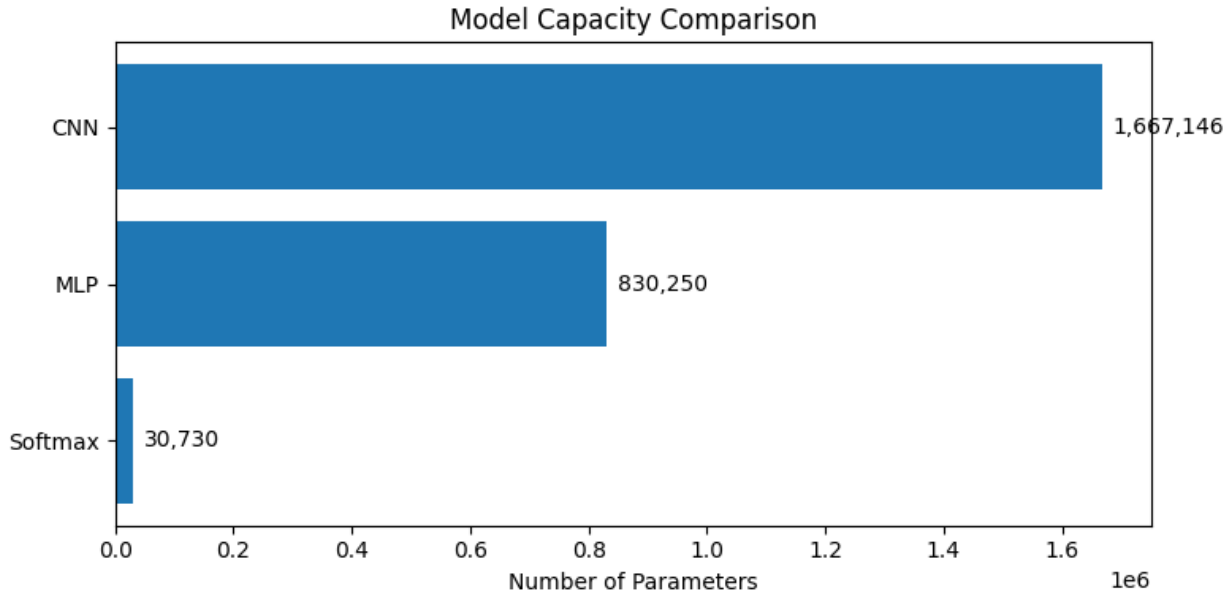


Figure 9. Model capacity of Softmax, MLP, CNN

3.3. Experimental result and analysis

3.3.1. Overral performance comparison

The results obtained after training three models on the same datasets are presented below; the best model was selected based on the highest accuracy on the validation set.

Metrics	Softmax	MLP	CNN
Train loss	1.5276	0.4048	0.01432
Validation loss	1.7795	1.3739	0.333584
Test loss	1.7799	1.3989	0.367557
Train accuracy (%)	48.97	86.9	99.96
Validate accuracy (%)	39.37	54.72	89.46
Test accuracy (%)	38.63	55.23	88.6
Marco Precision	0.3824	0.5529	0.88586
Marco Recall	0.3863	0.5523	0.886
Marco F1-score	0.3836272	0.5523526	0.88579
Trainable parameters	30730	830250	1667146
Training time (s)	2685.59	2806.75	3043.72

- ❖ Training and generalization performance

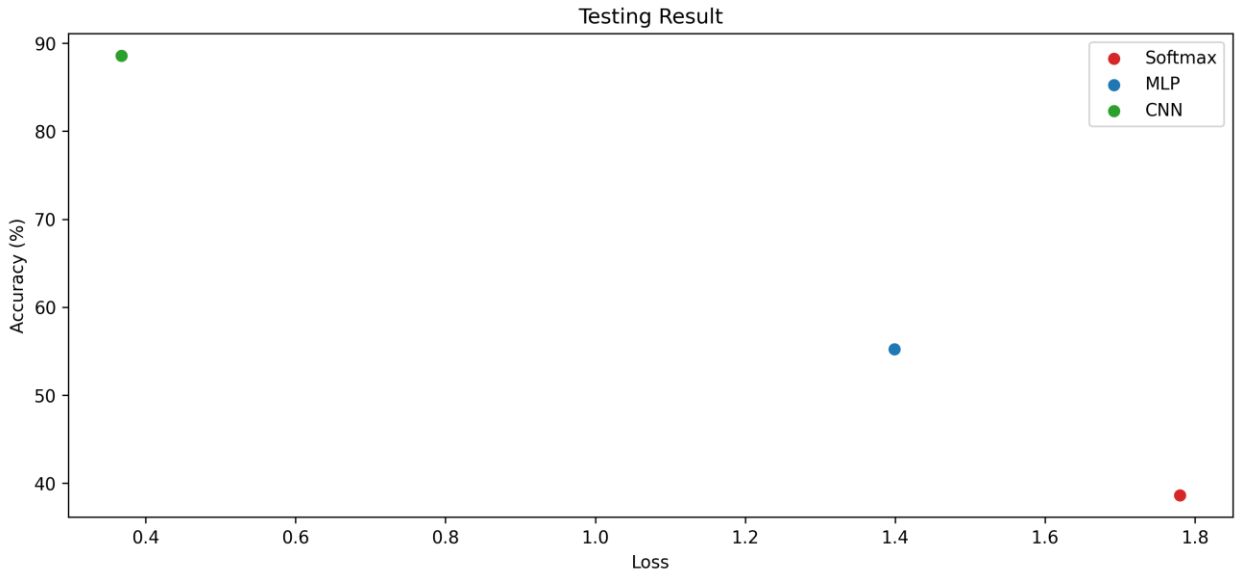


Figure 10. Loss and accuracy result on test set of Softmax, MLP, CNN

The Softmax classifier achieves a lowest accuracy and a highest loss on both set. This result indicate a limited learning capability because image classification requires capturing complex, non-linear patterns and spatial relationship which a linear classifier as itself cannot effectively learn.

A significant improvement compared to Softmax classifier, MLP perform a better result with 86.9% training accuracy, 54.72% validation accuracy and 55.23% test accuracy. Model start to capture more complex patterns with applied non-linear activation. However, the performance of the test dataset and validation set differs significantly from that of the training dataset. Model suggest overfitting should deepdive analyze the learning curve.

The CNNs model achieves the best results, with nearly 100% training accuracy (99.96%), 89.46% validation accuracy and 88.6% test accuracy. Its training loss is very closely to 0 while validation and test loss is significantly higher (0.3544 and 0.3670). Although the accuracy on the training set is very high, the gap between the performance on the training set and the validation set indicates that overfitting is occurring. However, the visualize results on the validation and test set suggest that this model provide a significantly better generalization capability compared to the other two models.

❖ Classification performance

The CNNs model achieves the highest Macro Precision (0.8857), Macro Recall (0.8857) and F1-score (0.8856). These value are closely aligned metrics indicate that the

model maintains a good balance between the reliability of its positive predictions and its ability to identify samples from each class.

Compare to CNN, MLP model achieves a Macro Precision of 0.5553, Macro Recall of 0.5534 and F1-score of 0.5536; Softmax classifier obtains a Macro Precision of 0.3857, Macro Recall of 0.3888 and F1-score of 0.3867. MLP model results indicate a moderate performance while Softmax model results indicate relatively low scores.

❖ Model capacity and training time

The Softmax classifier has 30730 trainable parameters as a simplest model. The MLP classifier has 830250 parameters, which is 27 times that of base model Softmax, while CNNs has 1667146 parameters which is 54 times that of base.

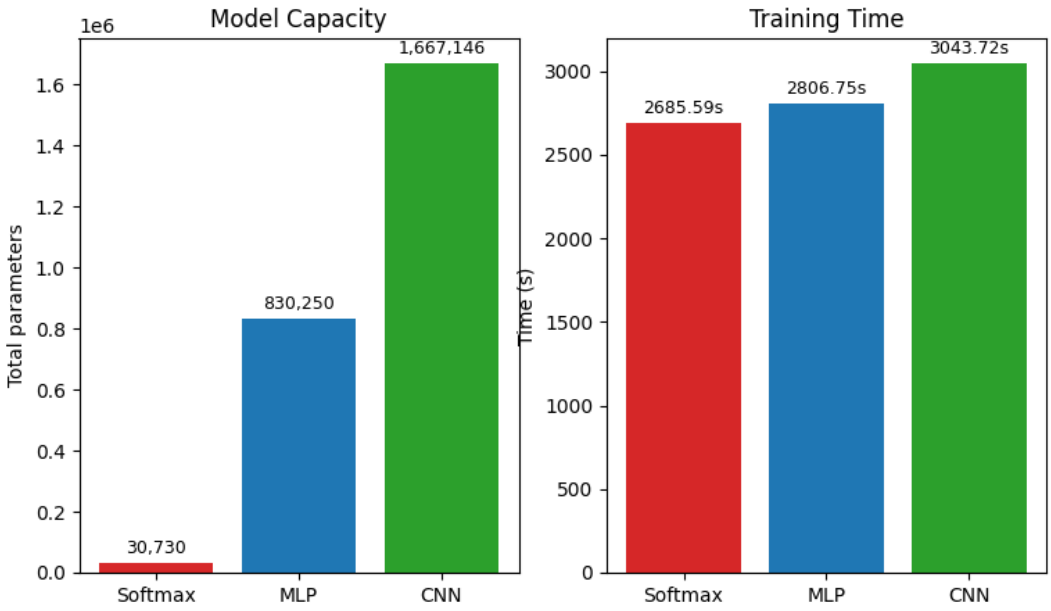


Figure 11. Model capacity and training time of Softmax, MLP, CNN

Although CNNs model has approximately twice as many trainable parameters as the MLP model, weight sharing allows convolutional layers to reuse the same filters across different spatial locations. This reduces the number of parameters compared with a fully connected architecture operating on similar image features. However, weight sharing does not necessarily reduce training time, as convolutional operations must still be performed across spatial locations. In thisresults, the CNNs model takes approximately 20.5% longer to train than the MLP (2,654.73 vs. 2,203.31 seconds), likely due to its computationally intensive convolutional operations. However, the CNNs model achieves substantially better

test accuracy (88.6% vs. 55.23%), indicating a better trade-off between training cost and classification performance in this experiment.

3.3.2. Training and validation analysis with loss/accuracy curve

The chart shows a clear improvement in performance following an upgrade to the network architecture.

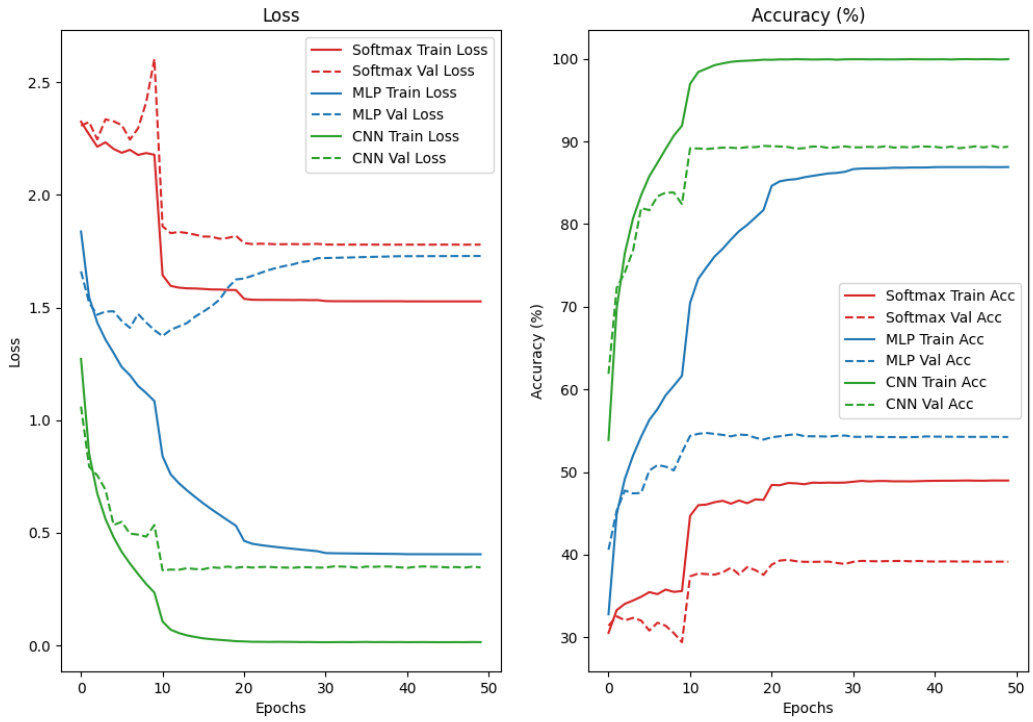


Figure 12. Loss/Accuracy curve of training/validation set by Softmax, MLP, CNN

Softmax model suggests an underfitting issue as both training and validation accuracies remain relatively low. From epoch 21, model have reached a plateau in training loss and accuracy, indicating that further training may not significantly improve its performance due to the limited representational capacity of a linear classifier.

Besides, MLP model introduces a serious overfitting. After epoch 10, validation loss begin to increase again (from 1.3625 to 1.7473) while training loss continues to decrease. The gap between train accuracy and validat accuracy is quite large. Possible explanation is that MLP perform flatten an image input into one-dimensional vector, thereby losing spatial structural information. This makes the perceptron network tends to memorize the training data rather than generalizing from it.

The CNNs model extracts spatial features from images extremely effectively with highest accuracy on both sets. Its training and validation losses stabilize after around 20 epochs indicate a quick convergence with a good performance. However, the gap between training and validation performance suggests some overfitting, as the validation loss plateaus after approximately 20 epochs and no longer improves significantly. Further parameter updates continue to reduce training loss but provide little to no improvement in validation performance. Consequently, further training becomes essentially pointless in terms of generalization capability.

3.3.3. Classification performance analysis

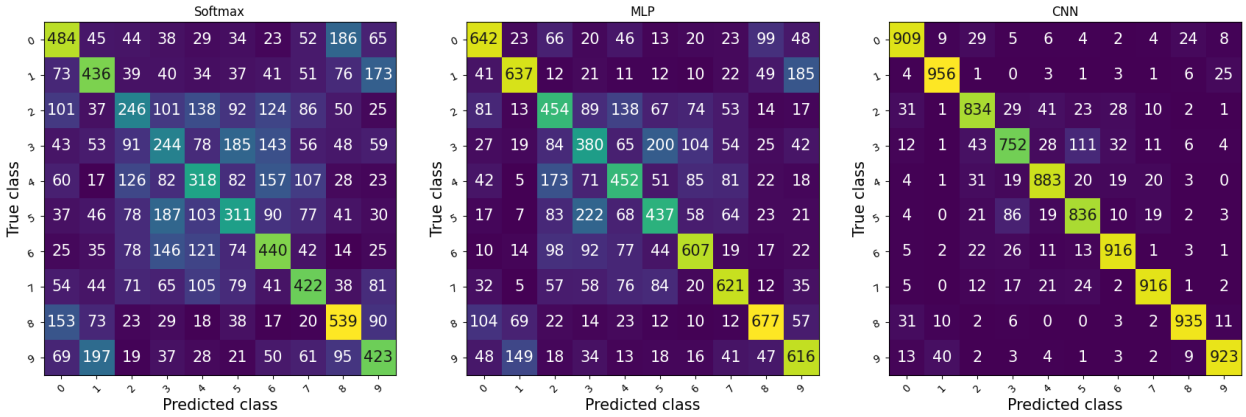


Figure 13. Confusion matrix comparison between Softmax, MLP, CNNs classifier

The classification performance of the three models (Softmax, MLP, and CNN) is evaluated using confusion matrices. Each confusion matrix contains 10 classes (labels 0–9), where the rows represent the true classes and the columns represent the predicted classes.

Model	Correct predictions	Misclassified samples	Accuracy
Softmax	3863	6137	38.63%
MLP	5523	4477	55.23%
CNN	8860	1140	88.6%

The result show a clear improvement in classification performance from Softmax to MLP to CNN. The CNNs achieves the highest accuracy of 88.6%, outperforming the others.

- The Softmax classifier achieves the lowest accuracy of 38.63%. The confusion matrix shows that many samples are incorrectly classified across different classes. These misclassifications indicate that the Softmax classifier has difficulty distinguishing between certain classes. This is expected result because the model uses a simple linear transformation followed by Softmax, without hidden layers to learn more complex feature representations.

- The MLP achieves an accuracy of 55.23%, which is an improvement of 16.6 percentage points over the Softmax classifier. The correct predictions improves for all classes indicate that the hidden layers of MLP enable the model to learn non-linear relationships. However, the remaining classifications suggests that the learned representations are still insuffient to distinguish some classes effectively.
- Most values in the confusion matrix are concentrated along the main diagonal, indicating that the CNNs correctly classifies the majority of samples in each class. In particular: class 1 (95.6% accuracy); class 8 (93.5% accuracy); class 6 and 7 (91.6% accuracy)... The lowest accuracy of classification class is class 3 with label "cat" – 75.2%.

3.3.4. Sample error analysis

Sample misclassifier of class 3 show:

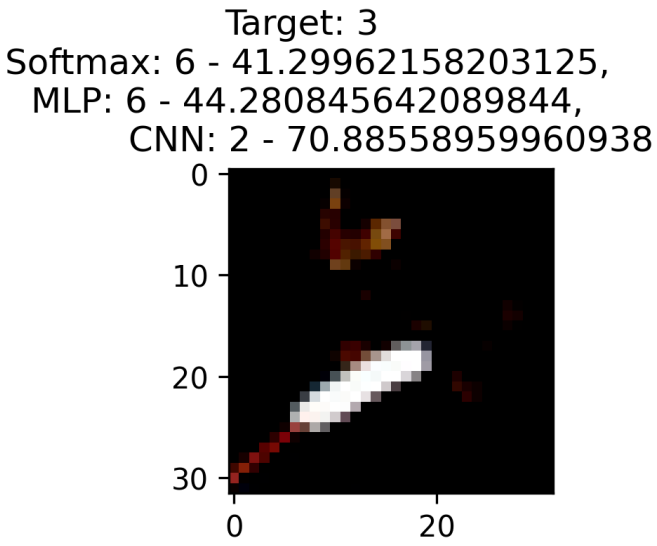


Figure 14. Error analysis of misclassifier image (target class: "cat")

The selected sample from Class 3 was incorrectly classified by all three models (Softmax, MLP, and CNN). Due to the small image size, visual details and distinctive features are difficult to distinguish. This may make the sample challenging to classify, even for the CNNs model.

Another sample, all models predict class "frog" instead of "dog":

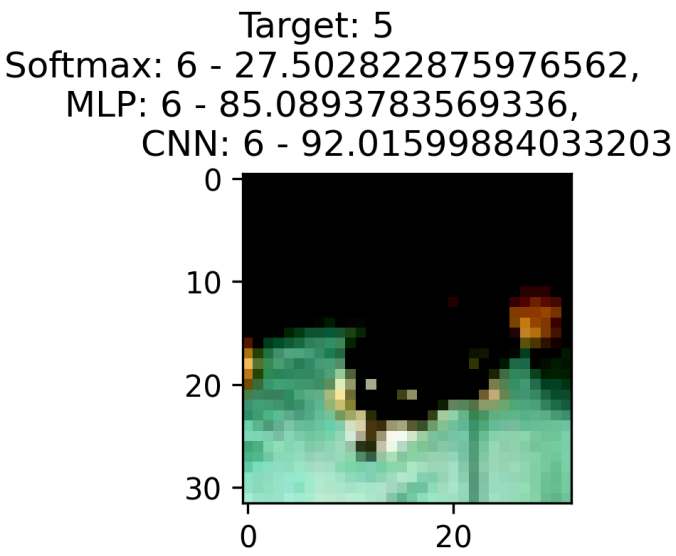


Figure 15. Error analysis of misclassifier image (target class: "dog")

In practice, small image resolution makes it difficult to distinguish between visually similar classes, such as dogs and frogs.

3.4. Discussion & key takeaways

The experimental results highlight the importance of model architecture in image classification performance. Different of architecture introduced difference of performance of accuracy and generation of model.

(1) Model complexity improves classification capability: hidden layer and nonlinear activation function allows the model to learn more complex relationships between input features. However, it remains limited because fuilly connected layer do not explicity exploit the spatial structure of images.
(2) Spatial feature extraction is important for image classification: Convilutional layer can capture local patterns such as edge, textures, shape… during training, while progressively learning more visual features. This ability to preserve and exploit spatial relationships makes CNNs more effective for image classification tasks that outperforming Softmax and MLP.
(3) Although CNNs achieves the best classification accuracy, some misclassifications remain, indicating opportunities for further improvement. Several approaches can be explored:
    - Data augmentation: apply random rotation, cropping, scalling and brightness adjustments to increase training data diversity and improve model generalization.
    - Regularization techniques: to achieve better performance and avoid overfitting, apply Dropout in some layer to disable randomly neural during training.

- Architecture optimization: experiment with deeper CNNs architecture, apply Residual Connections to improve feature extraction capabilities.
- Error analysis: identify potential issues such as visual similar image, noise impact or incorrect labels.

The improvements could further enhance classification accuracy and robustness of model. Howeverm their effectiveness should be verified through additional experiments on a validation set and evaluated on an independent test set.

Chapter 4. Experiment E2

4.1. Specific requirement

(1) Implement custom Multi-Head Self-Attention and compare with Pytorch MSA:
- Custom MSA: implement from basic operation (Linear, matrix multiplication, softmax, head concatenation), without using nn.MultiheadAttention or nn.TransormerEncoder.
- Pytorch MSA: using nn.MultiheadAttention, nn.TransormerEncoder

(2) Tokenization strategy:
- Patch-base tokens, optionally with CLS token.
- Pixel/row-based tokens.
- CNN-stem tokens (spatial feature map positions).

(3) Apply to Encoder model
- Integrate MSA into a lightweight image classifier consisting of a shallow encoder and classification head. LayerNorm, FFN (Feed-Forwar Neural Networks) and positional are optional.

(4) Evaluation
- Train and evaluate on the same dataset as Experiment E1.
- Compare custom MSA vs Pytorch MSA to evaluate correctness
- Compare different model with custom or Pytorch MSA and different tokenization methods base on accuracy, training/inference time and performance analysis.

4.2. Introduce attention mechanism

4.2.1. Objective

Alongside CNNs, Vision Transformer (ViT) models offer a new approach by applying the self-attention mechanism to image data, thereby provide an alternative method for modeling visual information. While CNNs extract local spatial features through

convolutional operations, ViT model capture the global relationships by employing a self-attention mechanism across image patches.

The objective of this experiment is to explore the application of Multi-Head Self-Attention (MSA) to image classification by implementing the mechanism from scratch and comparing it with PyTorch's built-in implementation. Furthermore, different image tokenization strategies are investigated to evaluate their effects on classification performance and computational efficiency.

4.2.2. Scaled Dot-Product Attention

In the self-attention mechanism, each input token is transformed into three different representations: Query (Q), Key (K), and Value (V). These representations serve different purposes in determining how information is exchanged between tokens.

- Query (Q): represent what information a token is looking for from other tokens
- Key (K): represent the characteristic of each token that are used to determine how relevant it is to a given query
- Value (V): represent a detailed information that a token contributes to other tokens when attention is computed

Self-Attention is implemented using Scaled Dot-Product Attention by three matrix: Query (Q), Key (K), Value (V), to calculate how strongly each token should attend to other tokems.

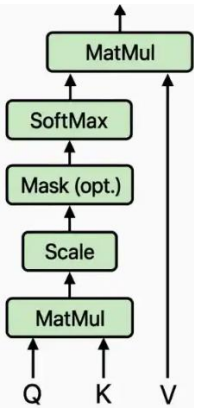


Figure 16. Scaled Dot-Product Attention formular

Matmul(Q,K) to calculate the similarity between Query and Key to determine the importance of each token relative to the others. Then scale it by divided by square root of d_k. These scaled similarity score are converted into attention weights using the Softmax function. Finally, matmul attention weights and Value are applied to produce the output – attention score. The attention score is computed as:

$$Attention - score(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where:

- Q, K, V: Q, K, V matrix
- d_k: dimension of the key vector
- mask (optional): after scale in operation $A = \frac{QK^T}{\sqrt{d_k}}$, if apply padding mask in this position, $A = -\infty$ then $softmax(-\infty) = 0$ introduces attention-score will be zero in padding tokens.

4.2.3. Multi-Head Self-Attention (MSA) overview

Limitation of Single-Head Attention show that it forced to average these potentially diverse relational signals into one representation. A single attention head can only learn one attention pattern at a time, limiting its ability to capture different types of relationship between tokens.

Multi-Head Attention addresses these limitations by using multiple attention heads that operate in different representation subspaces. Each head independently learns attention patterns, and their outputs are concatenated to produce a richer representation.

4.3. Model architecture

4.3.1. Multi-Head Self-Attention MSA

A Multi-Head Selt-Attention MSA show that:

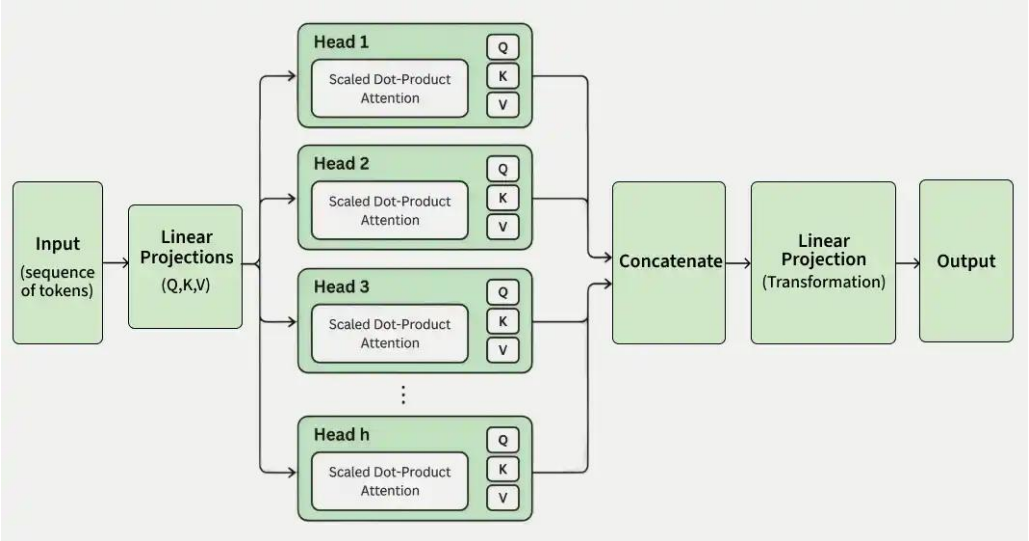


Figure 17. Multi-Head Self-Attention architecture

- Input (sequence of tokens): input of MSA should be tokens (not a raw image).
- Linear Projections (Q, K, V): three independent linear layer to project input embedding into Query (Q), Key (K), Value (V) representations. The weights and biases of all layers are updated during backpropagation.
- Head and Scaled Dot Production Attention: compute attention score only, no parameter be learned.
- Concatenate: Concatenate the outputs from all attention heads along the feature dimension to combine information from different representation subspaces. This is a tensor operation and does not introduce learnable parameters.
- Linear Projection (Transformation): apply a learnable linear layer to transform the concatenated representation back into the original embedding dimension. The weights and biases of all layers are updated during backpropagation.
- Output: produce the final MSA output with shape Batch*num_of_token*dimention. Each output token contains contextual information aggregated from other tokens through the attention mechanism. The output is passed to the subsequent Transformer Encoder components, such as residual connection, LayerNorm and FFN.

4.3.2. Transformer Encoder block

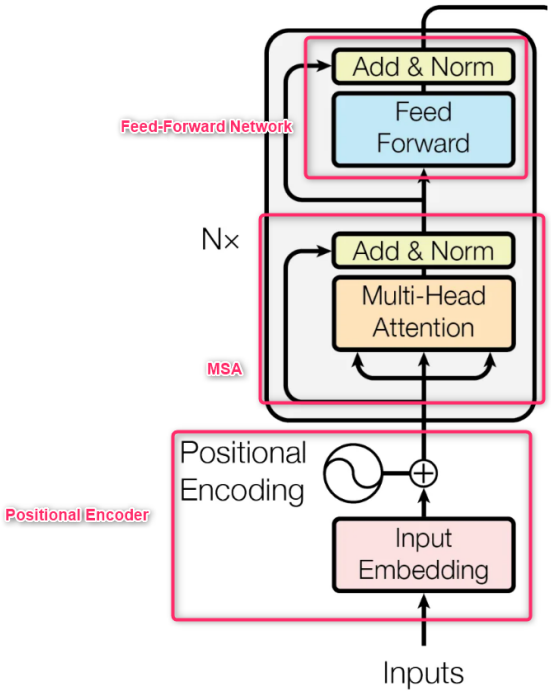


Figure 18. Transformer Encoder block architecture (PostNorm)

The encoder block of a transformer model consists of the following parts:

- Positional Encoder:
  + Input Embedding: convert input tokens into embedding vectors with a fixed dimension D. The embedding layer uses a learnable linear projection which updated weights and biases during backpropagation to map input features into the model embedding space.
  + Positional Encoding: add positional information to token embedding to preserve the spatial order of image patches/pixels. This leanable parameter is optional update during backpropagation or fixed value for encoding which has no trainable parameters.
  + Combine input embeddings with positional encodings through element-wise addition. There has no learnable parameters.
- Multi-Head Self-Attention:
  + MSA: apply to capture relationships between different tokens. Each attention head learns different representation subspaces. Q, K, V projections and output projection contain learnable parameters.
  + LayerNorm: normalize the MSA output to stabilize training and improve optimization. Learnable scale parameters is optional by enable elementwise affine.
  + Residual connection: this layer has no learnable parameters, only add the input to the MSA output to preserve information and facilitate gradient flow
- Feed-Forward Network:
  + FFN: apply two linear transformations with a non-linear activation function (e.g., GELU or ReLU) to each token independently. Expand the embedding dimension into a higher-dimensional hidden space, then project it back to the original dimension. Both linear layers contain learnable weights and biases.
  + LayerNorm: normalize the output of the FFN to maintain stable feature distributions. Contains learnable scale and bias parameters if affine normalization is enabled.
  + Residual connection: add the FFN output to its original input to preserve information and improve gradient propagation. There has no learnable parameters.

4.3.3. Image classifier design for experiment

Design and implement a custom Vision Transformer (ViT)-based image classifier to perform image classification on CIFAR-10. The architecture show as following:

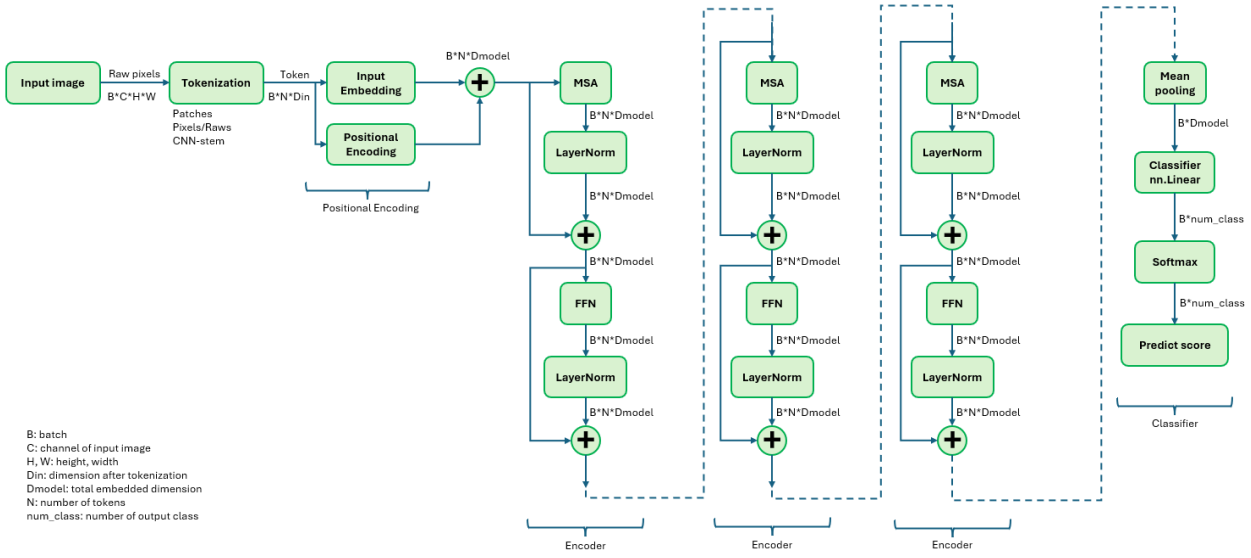


Figure 19. Full image classifier using Transformer Encoder block

- Input image: raw input of image (not a token)
- Tokenization: convert input images into sequence of token using different tokenization methods will be introduced (patch, pixel/row, CNNs extract feature map)
- Positional encoder consists of input embedding and positional encoding to represent token feature with spatial information
- Encoder block to learn contextualized image representations by combining information across tokens through Multi-Head Self-Attention (MSA) and refining token features through a Feed-Forward Network (FFN).
- Mean pooling: aggregate all token representations into a single feature vector for image classification
- Classifier and softmax: map aggregate value to predict scores for prediction

Re-use chapter 2 of general setup for training and evaluation pipeline. Review the following hyparameter should be apply:

batch_size	32
lr	0.01
mm	0.9
weight_decay	0.0005
num_epochs	70
step_size	30
gamma	0.1
num_worker	8
Loss function	Cross Entropy
Optimizer	SGD
Scheduler	StepLR

4.4. Tokenization Strategies

Tokenization is the process of converting an input image into a sequence of tokens that can be processed by the Transformer encoder. Unlike traditional CNNs, which directly process spatial feature maps through convolutional operations, Transformers require the input to be represented as a sequence of feature vectors.

In this experiment, 3 tokenization strategies are introduced: Patch Tokenization, Pixel Tokenization, and CNNs feature map Tokenization. Although these methods produce different numbers of tokens and feature representations, all token sequences are projected into the same embedding dimension ((d_{model}=128)) to ensure compatibility with the Transformer encoder.

4.4.1. Patch Tokenization

This tokenization strategy divides an input image into a grid of non-overlapping patches. Each patch is then flattened into a 1D-dimension vector and projected into embedding space using a learnable linear layer.

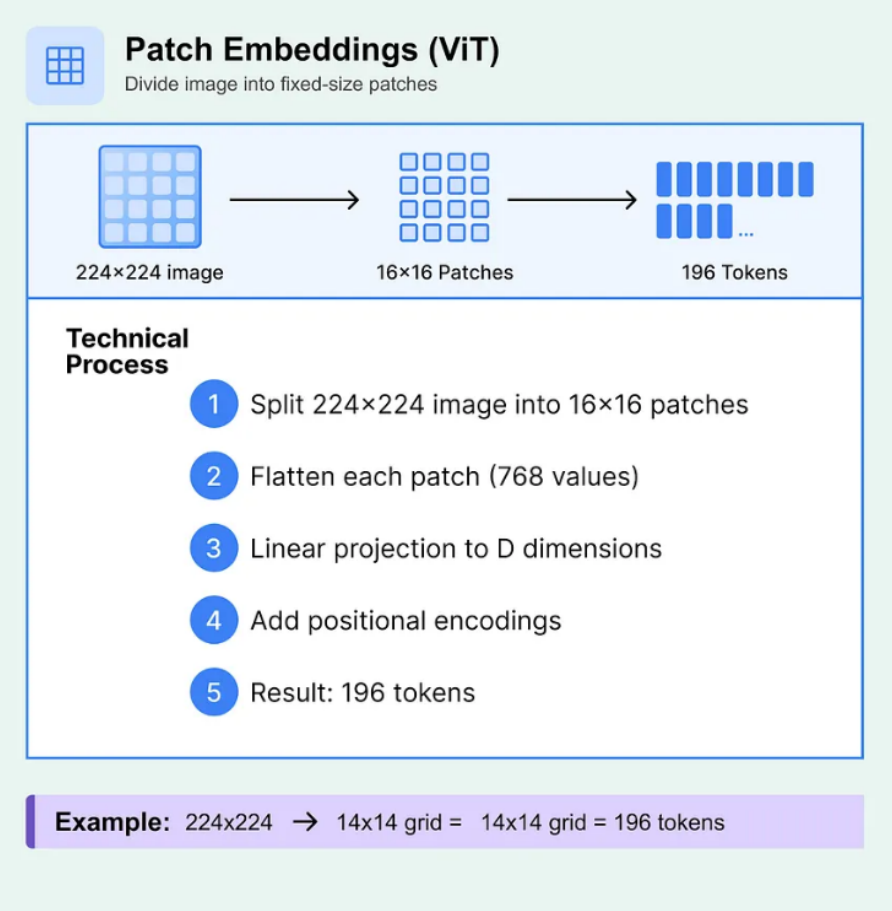


Figure 20. A 224×224 image with 16×16 patches yields 196 tokens (14×14 grid)

Positional embedding are subsequently added to preserve the spatial information of the patches. This method reduces the original image into a number of tokens while retaining local spatial information within each patch. However, fine-grained information may be lost when pixels are grouped into larger patches.

For input image of size 32*32 pixels, apply a grid patch 4*4 is selected to divide the image into 64 non-overlapping patches. This provides a balance between preserving local visual information and reducing the sequence length, making self-attention computationally more efficient while retaining sufficient spatial details for image classification.

4.4.2. Pixel/Rows Tokenization

Pixel Tokenization treats each individual pixel as a separate token. Each pixel is represented by its RGB values, forming a three-dimensional feature vector.

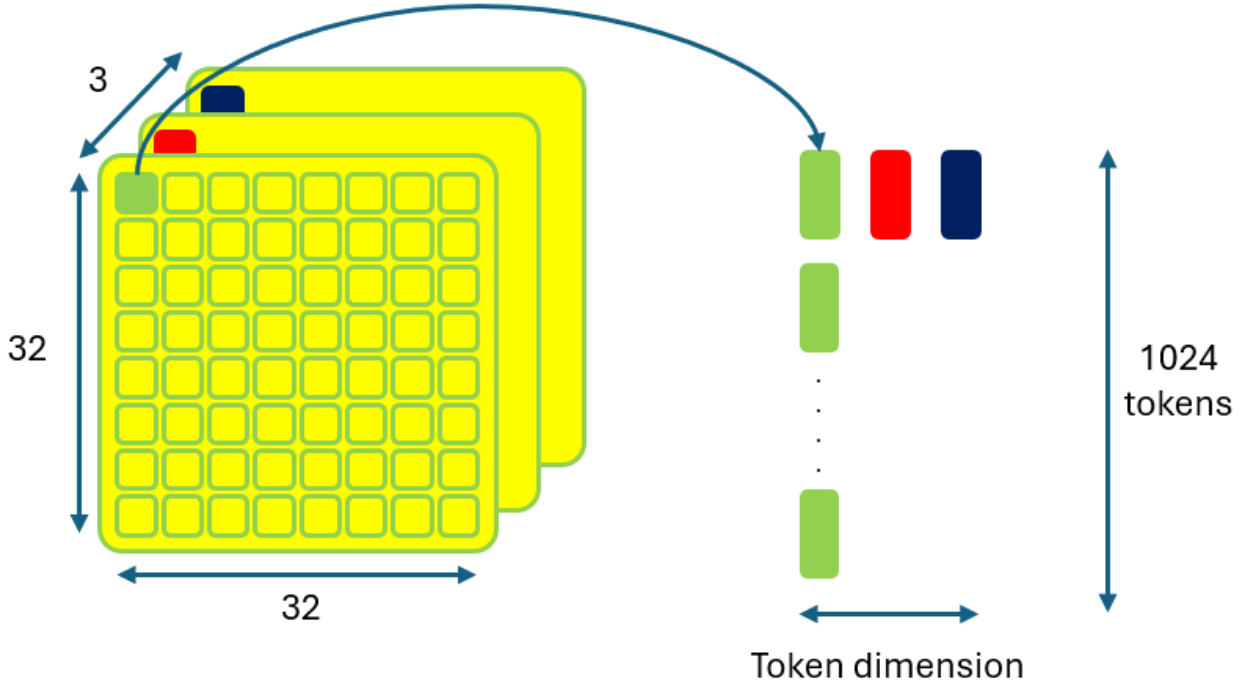


*Figure 21. A 32*32 image with pixel tokenization (1024 tokens * 3)*

Positional embeddings are then added to retain the original spatial arrangement of pixels. This method preserves fine-grained pixel-level information because no pixels are grouped or discarded during tokenization. However, it produces a significantly longer token sequence than Patch Tokenization, increasing the computational and memory requirements of self-attention.

For input image of size 32*32 pixels, pixel tokenization output 1024 token which is 16 times more tokens than Patch tokenization (4*4 grid patch – 64 tokens).

Same as pixel tokenization, each rows treats as a separate token.

4.4.3. CNNs feature map Tokenization

CNNs feature map Tokenization uses a CNNs model as a feature extractor before generating transformer tokens instead of directly converting raw image pixels into tokens, the CNNs first extracts higher-level visual features through convolutional operations.

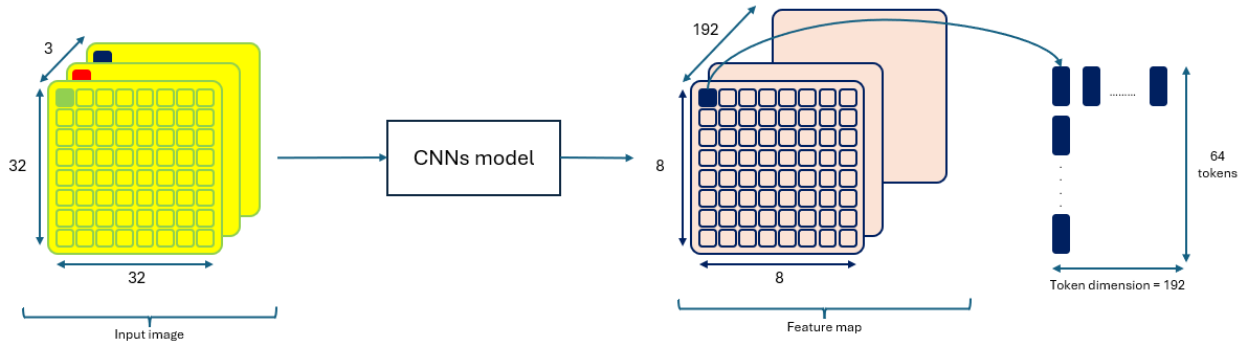


*Figure 22. A 32*32 image with CNNs extracted feature map tokenization*

Positional embeddings are added to preserve the spatial relationships between feature-map locations. The main advantage of this approach is that CNNs can extract local patterns and hierarchical visual features before applying self-attention. Consequently, the transformer operates on more informative feature representations rather than raw pixel values. However, the number of tokens and the amount of retained spatial information depend on the CNNs architecture and the spatial resolution of its output feature map.

4.5. Implementation details

4.5.1. Custom Tokenization

Create class of tokenization module help to select strategy of model from input.

(A) Patch tokenization (patch size = 4*4)

- Input image: batch_size*3*32*32
- Input of feature to MSA: 48
- Input of token to MSA: 64

```
class PatchesTokenization(nn.Module):
    def __init__(self, patches):
        super(PatchesTokenization, self).__init__()

        self.patches = patches

    def forward(self, x):
        self.x = x
        B, C, H, W = self.x.shape                                    (1)
```

Patch tokenization

(1) raw image input
(2) get input size
(3) create a grid patches along image (ensure non-overlap) by assert in class

```
        patches = []

        n = self.patches[0]                                    (2)
        m = self.patches[1]                                    (2)

        for h in range(0, H, n):
            for w in range(0, W, m):
                patches.append(self.x[:, :, h:h+n, w:w+m])     (3)

        patches = torch.stack(patches, dim=1)                  (4)
       patches = patches.reshape(B, -1, C * n * m)             (5)

        return patches
```

function which hide in report.
(4) Shape (B, num_patch, dim, patch, patch)
(5) Shape (B, num_tokens, dim)

(B) Pixels tokenization

- Input image: batch_size*3*32*32
- Input of feature to MSA: 3
- Input of token to MSA: 1024

```
class PixelTokenization(nn.Module):
    def __init__(self):
        super(PixelTokenization, self).__init__()

    def forward(self, x):
        self.x = x
        B, C, H, W = self.x.shape                              (1)
        pixels = self.x.permute(0, 2, 3, 1)                    (2)
        pixels = pixels.reshape(B, H * W, C)                   (3)
        return pixels
```

Pixel tokenization

(1) raw image input
(2) shape (B, H, W, C)
(3) shape (B, H*W, C) with H*W is number of tokens

(C) Rows tokenization

- Input image: batch_size*3*32*32
- Input of feature to MSA: 32
- Input of token to MSA: 96

```
class RowTokenization(nn.Module):
    def __init__(self):
        super(RowTokenization, self).__init__()

    def forward(self, x):
        self.x = x
        B, C, H, W = self.x.shape                              (1)
        pixels = self.x.permute(0, 2, 3, 1)                    (2)
        pixels_by_raw = pixels.reshape(B, H, C*W)              (3)
        return pixels_by_raw
```

Row tokenization

(1) raw image input
(2) shape (B, H, W, C)
(3) shape (B, H, C*W) with C*W is token dimension

(D) CNNs feature map tokenization

- Input image: batch_size*3*32*32
- Output of CNNs feature map: 192*8*8 (192 dimension) – use CNNs architecture from E1

- Input of feature to MSA: 192
- Input of token to MSA: 8*8 = 64

```
class CNNfeatureTokenization(nn.Module):
    def __init__(self, cnn_model: nn.Module):
        self.cnn_model = cnn_model                              (1)

    def extract_features(self, x):
        _, features = self.cnn_model(x)
        feat_to_token = features["block3"]                      (2)
        return feat_to_token

    def forward(self, x):
        feat_to_token = self.extract_features(x)                (2)
        B, C, H, W = feat_to_token.shape                        (3)
        tokens = feat_to_token.permute(0, 2, 3, 1)              (4)
        tokens = tokens.reshape(B, H * W, C)                    (5)
        return tokens
```

```
CNNstem tokenization

(1) use same CNNs
architecture from E1
(2) get output of
feature extraction
(3) shape (B, C, H,
W)
(4) shape (B, H, W,
C)
(5) shape (B, H*W,
C) with H*W is
number of tokens
```

Summary of both tokenization strategy:

Strategy	Patch	Pixel	Row	CNNstem
Input X	B, 3, 32, 32	B, 3, 32, 32	B, 3, 32, 32	B, 3, 32, 32
Number of token	64	1024	32	64
Dimension of token	48	3	96	192

Pixel tokenization generates the largest number of tokens (1024), significantly exceeding other strategies. This results in a much longer input sequence for MSA, leading to substantially higher computational and memory costs due to the quadratic complexity of self-attention with respect to sequence length.

Different tokenization strategies introduce different trade-offs between sequence length, feature representation, and computational complexity. CNN-stem tokenization provides richer feature representations, while pixel tokenization preserves the finest input-level information.

4.5.2. Custom projection & MSA

Custom implementation of Multi-Head Self-Attention using basic operations, including Linear projections, matrix multiplication, Softmax, and head concatenation, without using PyTorch's built-in nn.MultiheadAttention module.

First, Apply three learnable linear projections to the input tokens to generate Query, Key, and Value representations.

```
class CustomVectorQKV(nn.Module):
    def __init__(
        self,
```

```
(1) dimension of
each head
```

```
        in_features,
        d_model,
        num_heads,
        projection=CustomLinearProjection,
        proj_weights=None,
        bias=True
    ):
        super().__init__()

        self.d_head = d_model // num_heads                          (1)
        self.num_heads = num_heads
        self.in_features = in_features                              (2)

        self.proj = projection(
            in_features=in_features,
            out_features=d_model,
            fix_weights=proj_weights if proj_weights is not None else
None,
            bias=bias
        )                                                           (3)

    def forward(self, x):
        B, N, _ = x.shape                                           (4)
        x = self.proj(x) # (B, N, d_model)
        x_reshape = x.reshape(B, N, self.num_heads, self.d_head)  (5)
        x_transpose = x_reshape.transpose(1, 2)                     (6)
        return x_transpose
```

(2) input dimention of vector Q/K/V
(3) projection with fixed weights/bias if provided
(4) shape (B, N, in_features)
(5) shape (B, N, num_heads, d_head)
(6) shape (B, num_heads, N, d_head)

Second, implement Scaled Dot-Product Attention from above given Q, K, V.

```
class ScaledDotProductAttention(nn.Module):
    def __init__(self):
        super().__init__()
        self.softmax = nn.Softmax(dim=-1)

    def _get_score(self, q, k):
        t_qkt = torch.matmul(q, k.transpose(-2, -1)) / (self.d_head **
0.5)
        if self.mask is not None:
            t_qkt = t_qkt.masked_fill(self.mask == 0, float('-inf'))

        score = self.softmax(t_qkt)
        return score

    def _get_output(self, score):
        output = torch.matmul(score, self.v)
        return output

    def forward(self, q, k, v, mask=None):
        self.q = q
        self.k = k
        self.v = v
        self.d_head = self.q.size(-1)

        if mask is not None:
```

Implement same as formular in Chapter 4.2.2

Default mask is none that token can attention to all other tokens

```
            self.mask = mask
        else:
            self.mask = None

        score = self._get_score(self.q, self.k)
        output = self._get_output(score)

        return output
```

Then, implement custom MSA use this class to calculate Q, K, V vector.

```
class customMHA(nn.Module):
    import torch.nn as nn

    def __init__(self, in_features, d_model, num_heads,
attention=ScaledDotProductAttention, bias=True, fixed_weights=None,
projection=CustomLinearProjection, proj_weights=None):
        super(customMHA, self).__init__()

        self.d_model = d_model
        self.num_heads = num_heads
        self.proj = projection
        self.bias = bias
        self.fixed_weights = fixed_weights                    (1)
        self.proj_weights = proj_weights                      (1)
        self.in_features = in_features
        self.d_head = d_model // num_heads
        self.attention = attention()                          (2) (4)

        # Initialize Q, K, V projections
        self.q_proj = CustomVectorQKV(                        (3)
            in_features=in_features,
            d_model=d_model,
            num_heads=num_heads,
            projection=self.proj,
            proj_weights={
                "weights": fixed_weights["q_weight"],
                "bias": fixed_weights["q_bias"] if bias else None
            } if fixed_weights is not None else None,
            bias=bias
        )

        self.k_proj = CustomVectorQKV(                        (3)
            in_features=in_features,
            d_model=d_model,
            num_heads=num_heads,
            projection=self.proj,
            proj_weights={
                "weights": fixed_weights["k_weight"],
                "bias": fixed_weights["k_bias"] if bias else None
            } if fixed_weights is not None else None,
            bias=bias
        )

        self.v_proj = CustomVectorQKV(                        (3)
```

(1) using a fixed weights and biases of token projection and output projection to compare the attention score with Pytorch MSA
(2) assign custom Scaled Dot-Product Attention implemented in above
(3) calculate vector Q, K, V
(4) calculate attention score
(5) concatenate the outputs of all attention heads along the feature dimension
(6) projection output layer

```
            in_features=in_features,
            d_model=d_model,
            num_heads=num_heads,
            projection=self.proj,
            proj_weights={
                "weights": fixed_weights["v_weight"],
                "bias": fixed_weights["v_bias"] if bias else None
            } if fixed_weights is not None else None,
            bias=bias
        )

        # Initialize output projection
        self.out_proj = self.proj(
            in_features=d_model,
            out_features=d_model,
            fix_weights=proj_weights if proj_weights is not None else
None,
            bias=bias
        )

    def _get_query(self, x):
        return self.q_proj(x)

    def _get_key(self, x):
        return self.k_proj(x)

    def _get_value(self, x):
        return self.v_proj(x)

    def _get_attention_score(self, q, k, v):                    (5)
        output = self.attention(q, k, v)
        return output

    def _concat_output(self, output):
        batch_size, num_heads, seq_len, d_head = output.size()

        output = output.transpose(1, 2).contiguous()

        output = output.view(
            batch_size,
            seq_len,
            num_heads * d_head
        )
        return output

    def _get_output_proj(self, output):
        return self.out_proj(output)

    def forward(self, x): projections using fixed weights if provided
        q = self._get_query(x)                                  (3)
        k = self._get_key(x)                                    (3)
        v = self._get_value(x)                                  (3)
        attn_score_heads = self._get_attention_score(q, k, v)   (4)
        attn_output = self._concat_output(attn_score_heads)     (5)
        attn_output_proj = self._get_output_proj(attn_output)   (6)
```

`return attn_output_proj`	

4.5.3. Custom Transformer Encoder

Custom a transformer block consists of MSA, Residual Connection and LayerNorm, FFN, Residual Connection and LayerNorm,

```
class customTransformerBlock(nn.Module):
    def __init__(self, multihead_block, d_model, d_ff):
        super(customTransformerBlock, self).__init__()
        self.d_model = d_model
        self.d_ff = d_ff
        self.mha = multihead_block
        self.norm1 = nn.LayerNorm(self.d_model)
        self.ffn = nn.Sequential(
            nn.Linear(self.d_model, self.d_ff),
            nn.ReLU(),
            nn.Linear(self.d_ff, self.d_model)
        )
        self.norm2 = nn.LayerNorm(self.d_model)

    def forward(self, x):

        # Multi-Head Self-Attention
        attn_output = self.mha(x)
        assert attn_output.shape == x.shape

        # Residual + LayerNorm (1)
        x = self.norm1(x + attn_output)

        # Feed Forward Network
        ffn_output = self.ffn(x)

        # Residual + LayerNorm (2)
        y = self.norm2(x + ffn_output)

        return y
```

Implement detailed along to design of block

Build a transformer encoder block from transformer block in above.

```
class customTransformerEncoder(nn.Module):
    def __init__(self, transfomer_block, num_layers, multihead_attn,
d_model, d_ff):
        super().__init__()
        self.num_layers = num_layers
        self.d_model = d_model
        self.d_ff = d_ff                                          (1)
        self.transfomer_block = transfomer_block
        self.encoder_layer = nn.ModuleList([                      (2)
            self.transfomer_block(multihead_attn(), self.d_model,
self.d_ff) for _ in range(self.num_layers)
        ])

    def forward(self, x):
```

(1) hidden dimension of FFN
(2) create sequential of transformer block

```
        for layer in self.encoder_layer:
            x = layer(x)
        return x
```

4.5.4. Custom Image Classifier

The custom image classifier converts input images into tokens, applies embedding and positional encoding, extracts contextual features through a Transformer encoder, aggregates token representations using mean pooling, and performs classification through a fully connected layer.

```
class customImageClassifier(nn.Module):
    def __init__(self, tokenizer, embedding_layer, positional_layer,
transformer_block, d_model, num_classes):
        super().__init__()
        self.tokenizer = tokenizer
        self.embedding_layer = embedding_layer
        self.positional_layer = positional_layer
        self.transformer = transformer_block
        self.classifier = nn.Linear(d_model, num_classes)

    def forward(self, images_input):
        # Tokenization
        x = self.tokenizer(images_input)                          (1)

        # Embedding x using the embedding layer
        x = self.embedding_layer(x)                               (2)

        # Positional encoding
        x = self.positional_layer(x)                              (3)

        # Transformer
        x = self.transformer(x)                                   (4)

        # Token aggregation (mean pooling over token dimension)
        x = x.mean(dim=1)                                         (5)

        # Classification (basic classification layer)
        logits = self.classifier(x)                               (6)

        return logits
```

(1) Tokenization method apply
(2) project token into the embedding space of MSA
(3) inject positional information into token embedding
(4) transformer encoder
(5) apply mean pooling across the token dimention
(6) map the aggregated representation to class logits

4.6. Experimental result and analysis

4.6.1. Custom MSA vs Pytorch Native MSA

Use the same projection weights and biases for both custom MSA and PyTorch MSA to ensure a fair comparison of their attention scores.

The attention score output from custom MSA is the same as Pytorch MSA for both tokenization method. The evidence of patch tokenization can be observed that:

```
[38]  1  pytorch_output
      ✓ 0.0s

...   tensor([[[-30.4248, -12.4866,  10.6228,  ...,  54.7190,  40.0436,  20.7242],
               [-22.9785,  -3.1860,  12.9488,  ...,  53.5585,  38.3366,  24.6296],
               [-22.3350,  -4.4065,  14.1241,  ...,  53.8383,  39.2758,  27.2043],
               ...,
               [-23.2840,  -5.1803,  10.6289,  ...,  52.7062,  37.7203,  26.5221],
               [-21.3451,  -9.9292,   4.2858,  ...,  54.7523,  38.0375,  28.7492],
               [-21.5186, -10.7317,  10.6727,  ...,  52.3892,  39.9114,  28.7851]]],
             device='cuda:0', grad_fn=<TransposeBackward0>)

[39]  1  output
      ✓ 0.0s

...   tensor([[[-30.4248, -12.4866,  10.6228,  ...,  54.7190,  40.0436,  20.7242],
               [-22.9785,  -3.1860,  12.9488,  ...,  53.5585,  38.3366,  24.6296],
               [-22.3350,  -4.4065,  14.1241,  ...,  53.8383,  39.2758,  27.2043],
               ...,
               [-23.2840,  -5.1803,  10.6289,  ...,  52.7062,  37.7203,  26.5221],
               [-21.3451,  -9.9292,   4.2858,  ...,  54.7523,  38.0375,  28.7492],
               [-21.5186, -10.7317,  10.6727,  ...,  52.3892,  39.9114,  28.7851]]],
             device='cuda:0')

[41]  1  assert torch.allclose(output, pytorch_output, atol=1e-6)
      2  print("[PASS] Output match test")
      ✓ 0.0s

...   [PASS] Output match test
```

Figure 23. Attention score of custom MSA and Pytorch MSA be (patch tokenization with fixed weights/biases for both)

For other methods, the given result met expected that attention score is the same.

4.6.2. Performance analysis base on different tokenization strategies

4.6.2.1. Overall performance comparison

Apply different tokenization strategies to the same image classifier architecture to evaluate their impact on classification performance and computational efficiency.

Metrics	Patch	Pixel	Row	CNNstem
Train loss	0.03472	0.4280686	0.0122267	0.00076416
Validation loss	0.8085	0.966346	1.0336	0.441518
Test loss	1.2395	1.1457	1.7914	0.5321
Train accuracy (%)	99.6675	84.79	99.975	100
Validate accuracy (%)	74.22	67.23	67.03	88.97
Test accuracy (%)	70.60	63.73	63.70	88.50

Number of tokens	64	1024	32	64
Marco Precision	0.7028	0.636425	0.6308547	0.8847542
Marco Recall	0.706	0.6373	0.637	0.88500005
Marco F1-score	0.7037037	0.63625	0.632645	0.8847267
Trainable parameters	413194	530314	415242	2098772
Training time (s)	4470.15	7 hour	3414.40	4138.18
Testing time (s)	16	39.49	18.64	16.788
Epoch	70	53 then interrupt	70	70

Overall, CNN-stem achieved the best classification performance, with 88.50% test accuracy and a macro F1-score of 0.8847, outperforming the other three tokenization strategies. It also obtained the highest validation accuracy (88.97%) and the lowest validation loss (0.4415), showing strong performance on both validation and test sets. In terms of computational cost, Pixel tokenization required the longest training time (approximately 7 hours) because it generated 1,024 tokens per image. In contrast, Patch and CNN-stem required approximately 74.5 minutes and 69 minutes, respectively. CNN-stem achieved the best classification performance with a relatively reasonable training and testing time, although it had the highest number of trainable parameters.

❖ Training and generalization performance

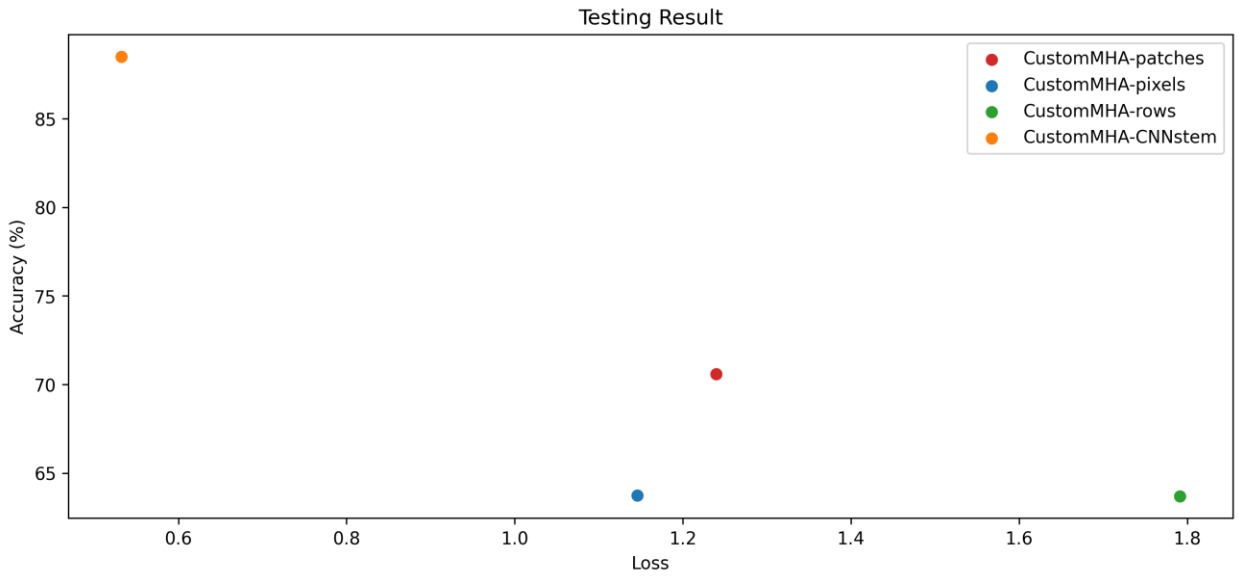


Figure 24. Loss and accuracy result on test set of both tokenization method

- Patch: train accuracy 99.67%, train loss 0.0347 indicating that the model successfully learned the training samples. But its validation accuracy decreased to 74.22%, test accuracy get more lower to 70.60%. This gap between training and evaluation performance suggests overfitting of model. However, Patch achieved the

second-highest test accuracy and macro F1-score (0.7037), showing better generalization than Pixel and Row.

- Pixel: take a longest time of training (53 epoch instead 0f 70 epoch but time reaches 7 hours) but testing and validation accuracy not have a significant improvement. Although the gap between training and test accuracy was smaller than that of the other strategies, the overall classification performance remained relatively low. This suggests that representing each pixel as an individual token did not provide sufficiently effective representations for the classification task.
- Row: achied an almost perfect training accuracy of 99.98%. However, its validation accuracy dropped significantly to 67.03%, while test accuracy reached only 63.70%. Same as Patch tokenization, the overfitting is introduced. Compared with Patch, Row achieved a higher training accuracy but lower validation and test accuracies, suggesting that learning from row-level representations did not translate into better generalization. One possible explanation is that aggregating pixels by rows may lose fine-grained spatial information that is important for distinguishing image classes.
- CNN-stem: archived 100% training accuracy and a validation accuracy of 88.97%, the highest among all strategies. Its validation loss was also the lowest at 0.4415. More importantly, it maintained a high test accuracy of 88.50%, with a macro F1-score of 0.8847. The small gap between validation and test accuracy suggests that CNN-stem generalizes well to unseen images. Although its training accuracy is higher than its validation and test accuracy, its overall performance indicates that the CNNs feature extraction stage helps produce more useful representations for image classification.

So, with the small dataset (less than 100k image), the models can learn the training images very well but may not perform as well on new, unseen images. This is particularly clear in Patch and Row, where training accuracy is almost 100% but test accuracy is considerably lower. In contrast, CNN-stem maintains high performance on both validation and test sets, suggesting that combining CNNs feature extraction with self-attention helps the model learn more generalizable image features. Using more training images and data augmentation could further improve generalization.

❖ Model capacity and training time

The number of tokens has a direct impact on the computational cost of the self-attention mechanism as following:

- Patch: provides a relatively good balance between computational efficiency and classification performance. Its testing time is 16 seconds, the lowest among the four methods. The training time take longer than CNN-stem and row tokenization.

- Pixel: introduce the highest computational cost due to its 1024 token sequence length. Its training takes approximately 7 hours then terminate, while testing takes 39.49 seconds. However, training for a long time but the test accuracy not take the best value.
- Row: reduces the sequence length compared with Pixel, resulting in a shorter training time of 3414.4 seconds. However, this reduction does not translate into better classification performance.
- CNN-stem: achieves the highest validation accuracy while maintaining a relatively low computational cost. Its testing time is 16.788 seconds, close to Patch tokenization while the trainable parameter is highest.

Overall, these results demonstrate a trade-off between sequence length, model complexity, computational cost, and classification performance. A longer sequence or longer training time does not necessarily lead to better results. CNN-stem provides the best classification performance, while Patch offers a simpler model with relatively low computational cost.

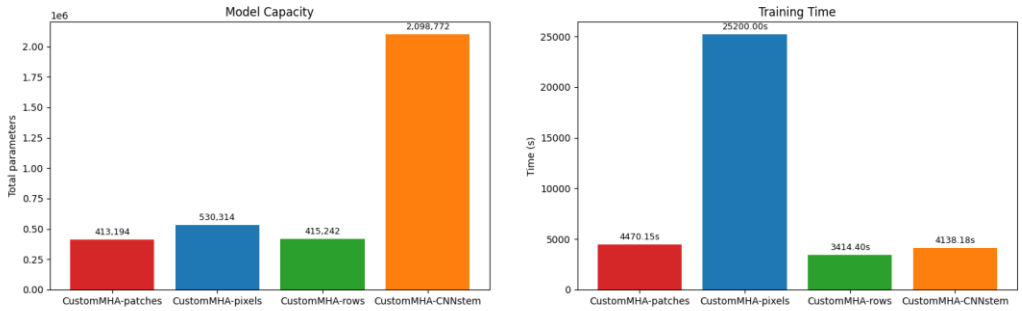


Figure 25. Model capacity and training time of both tokenization method

4.6.2.2. Training and validation analysis with loss/accuracy curve

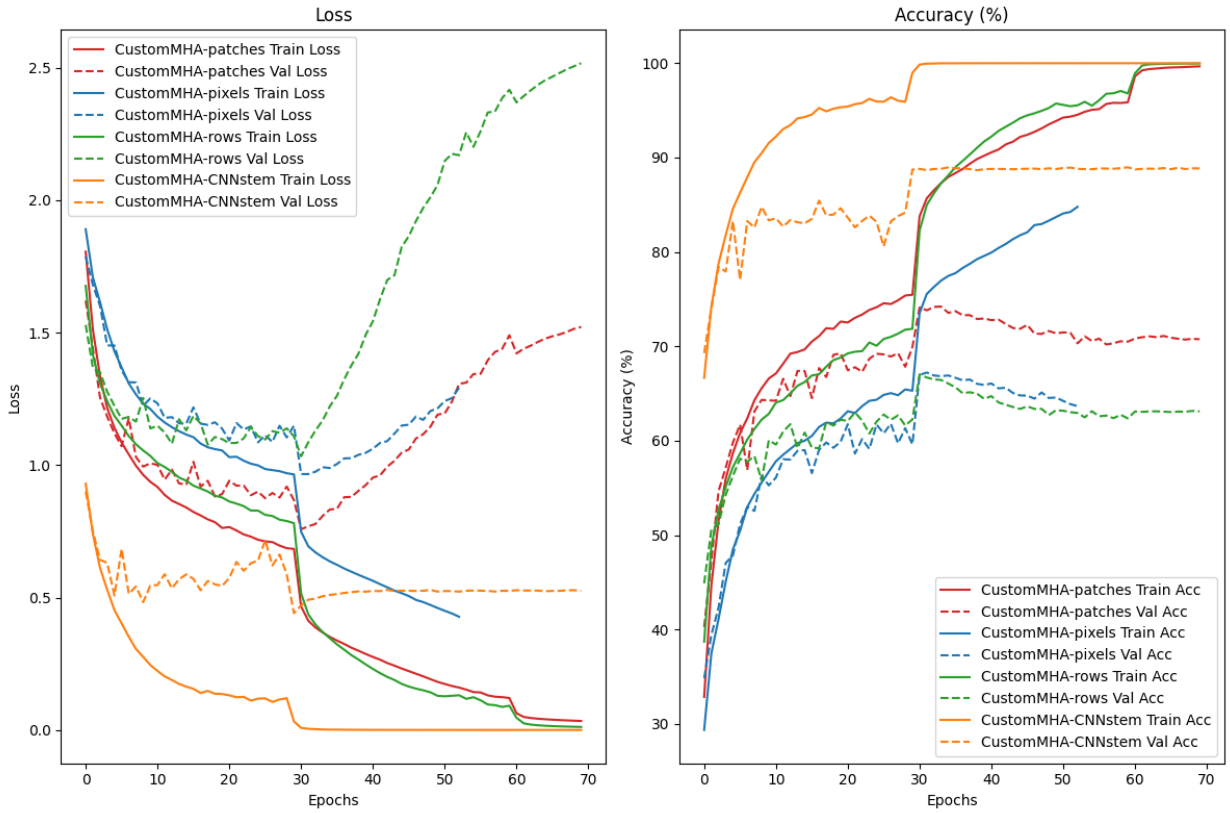


Figure 26. Loss/Accuracy curve of training/validation set by both tokenization method

From Patch tokenization's curve, training loss continuously decreases and training accuracy reaches nearly 100%. However, after approximately epoch 30, the validation loss starts increasing while validation accuracy gradually decreases. This indicates that the model learns the training data very well but becomes less effective at predicting unseen samples. Therefore, Patch tokenization shows signs of overfitting.

Pixel tokenization has a slower learning process compared with the other strategies. Its training accuracy reaches only around 85%, while validation accuracy remains around 60–70%. After approximately epoch 30, the validation loss increases and validation accuracy gradually declines. This suggests that the model struggles to learn effective representations from individual pixels and also shows signs of overfitting.

Row tokenization achieves almost 100% training accuracy, with training loss approaching zero. However, its validation accuracy remains around 63–67%, while validation loss increases significantly after approximately epoch 30. The large gap between training and validation performance indicates severe overfitting. The model appears to

memorize the training samples rather than learn features that generalize well to unseen images.

CNN-stem achieves nearly 100% training accuracy relatively early. Its validation accuracy also reaches approximately 89% and remains relatively stable, while validation loss stays around 0.5. Compared with Patch and Row, CNN-stem shows more stable validation performance throughout training. Although a small gap remains between training and validation accuracy, its stable validation curve and high test accuracy suggest better generalization.

For all tokenization methods, overfitting is very clear for Patch and Row, CNNs has the most stable training but it suggest a small overfitting and provides the best generalization performance among the four strategies. Therefore, combining CNN-based feature extraction with MSA appears to be more effective for this image classification task.

4.6.2.3. Classification performance analysis

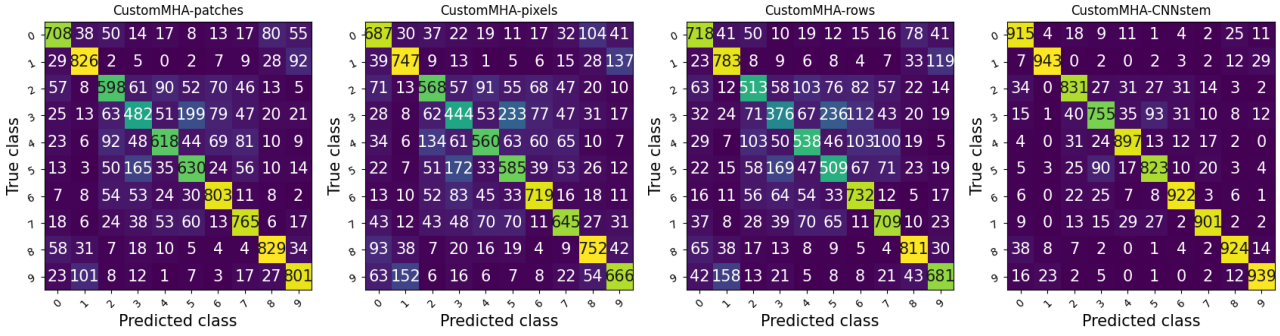


Figure 27. Confusion matrix comparison between both tokenization method

The classification performance of all model is evaluated using confusion matrices. Each confusion matrix contains 10 classes (labels 0–9), where the rows represent the true classes and the columns represent the predicted classes.

Model	Correct predictions	Misclassified samples	Accuracy
Patch	7060	2940	70.6%
Pixel	6373	3627	63.73%
Row	6370	3630	63.7%
CNN-stem	8850	1150	88.50%

The results show a clear improvement in classification performance across the four tokenization strategies. CNN-stem achieves the highest accuracy of 88.50%, outperforming Patch, Pixel, and Row.

Patch correctly classify 7060 out of 10000 samples. The confusion matrix shows that most correct predictions are concentrated along the main diagonal. However, some classes are still frequently confused, particularly classes 3 and 5. This indicates that Patch tokenization can capture useful image features but still has difficulty distinguishing between certain classes.

Compare with Patch, Pixel produces more misclassifications across different classes. In particular, classes 3 and 5 are frequently confused. This suggests that treating each pixel as an individual token does not provide sufficiently effective feature representations for distinguishing image categories.

Almost identical to Pixel, Row's confusion matrix also shows considerable misclassification between several classes, especially classes 3 and 5. Despite reducing the sequence length compared with Pixel, Row tokenization does not provide a meaningful improvement in classification performance.

CNN-stem achieves the highest accuracy of 88.50%, correctly classifying 8,850 out of 10,000 samples, with only 1,150 misclassified samples. Most predictions are concentrated along the main diagonal, indicating that the model correctly classifies the majority of samples across different classes. Compared with the other strategies, CNN-stem produces substantially fewer misclassifications, suggesting that CNNs feature extraction helps the model capture more useful spatial features before applying self-attention.

4.6.2.4. Sample error analysis

Sample misclassifier of class 3 show:

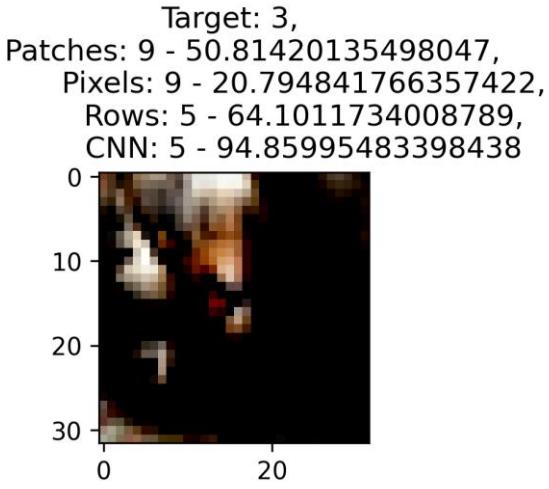


Figure 28. Error analysis of misclassifier image (target class: "cat")

Another sample, all models predict other classes instead of “dog”:

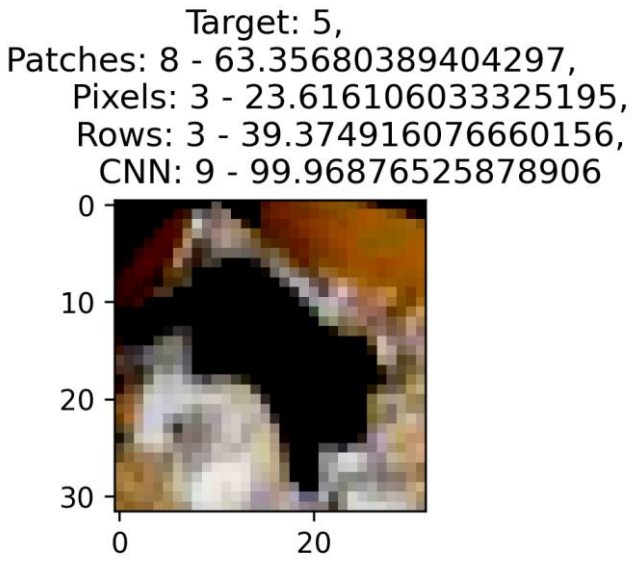


Figure 29. Error analysis of misclassifier image (target class: "dog")

The selected sample from Class 3 and Class 5 (almost incorrect classifier class) was incorrectly classified by all 4 models with different tokenization strategy. Due to the small image size, visual details and distinctive features are difficult to distinguish. This may make the sample challenging to classify, even for the CNNs model.

4.7. Discussion & key takeaways

The experimental results highlight the importance of tokenization strategies and model architecture (compare with E1) in image classification performance. Different tokenization methods introduce differences in classification accuracy, computational cost, and the model's ability to generalize to unseen images.

(1) Token representation affects classification performance: the way an image is converted into tokens has a significant impact on the effectiveness of MSA. CNN-stem achieves the highest test accuracy of 88.50%, followed by Patch (70.60%), Pixel (63.73%), and Row (63.70%). This indicates that providing meaningful feature representations to the attention mechanism is more important than simply increasing the number of tokens.

(2) Combining CNNs and MSA improves feature extraction: CNNs can capture local patterns such as edges, textures, and shapes during feature extraction, while MSA captures relationships between different spatial regions. Combining these two mechanisms allows the model to exploit both local visual features and global relationships, making CNN-stem more effective for this image classification task.

(3) Increasing the number of tokens introduces higher computational costs: Pixel tokenization generate 1024 tokens from a 32*32 image, while Patch tokenization with a 4*4 patch size generates only 64 tokens. This lead to Pixel requires significantly more computation. However, its longer training time does not translate into better classification performance. This suggests that a larger number of tokens does not necessarily provide more useful information.

(4) Generalization remains a challenge: Patch and Row show clear signs of overfitting, as their training accuracy approaches 100% while validation and test accuracy remain considerably lower. Even though CNNs introduces a stable training, but it also introduce a small overfitting.

(5) Although CNN-stem achieves the best classification performance, some misclassifications remain, indicating opportunities for further improvement. Several approaches can be considered:

- Data augmentation: apply random rotation, cropping, scaling, and brightness adjustments to increase training data diversity and improve the model's ability to recognize unseen images.
- Regularization techniques: apply Dropout, weight decay, and early stopping to reduce overfitting and prevent the model from relying too heavily on specific training samples.
- Architecture optimization: experiment with different patch size, embedding dimension, and numbers of attention heads. Deeper CNNs architecture or Residual Connection could also be explored to improve feature extraction capabilities.
- Tokenization optimization: investigate overlapping patches or different CNNs output feature-map sizes to preserve more useful spatial information while controlling the number of tokens.
- Error analysis: analyze misclassified samples to identify potential issues, such as visually similar class, image noise, insufficient feature extraction, or incorrect labels.

These improvements could further enhance classification accuracy and model robustness. However, their effectiveness should be verified through additional experiments on a validation set and evaluated on an independent test set.

Chapter 5. Experiment E3

5.1. Specific requirement

(1) Implement LSTM and GRU for image classification:

- LSTM: implement using nn.LSTM
- GRU: implement using nn.GRU

- Models must process image sequence and output predictions for same data with E1.
- Use the same recurrent architecture configuration for a fair comparison.

(2) Tokenization Strategy:

- Patch-base tokens, optionally with CLS token.
- Pixel/row-based tokens.
- CNN-stem tokens (spatial feature map positions).

(3) Apply to image classification model:

- Integrate LSTM or GRU into a lightweight image classifier.

(4) Evaluation:

- Train and evaluate LSTM and GRU on the same dataset and split as Experiment E1.
- Compare LSTM vs. GRU using the same tokenization strategy and comparable hyperparameters.
- Compare recurrent models against the non-recurrent MLP/CNNs baselines from E1.
- Evaluate based on classification accuracy, number of trainable parameters, training/inference time, and performance analysis.
- Discuss the ability of LSTM and GRU to capture sequential dependencies between image tokens compared with MLP/CNN.

5.2. Sequential modeling pipeline

5.2.1. Recurrent neural networks

Recurrent Neural Networks (RNNs) are neural network architectures designed to process sequential data by maintaining a hidden state that captures information from previous time steps. Unlike traditional feed-forward neural networks, which process inputs independently, RNNs incorporate information from previous steps when processing the current input. For image classification, an image can be transformed into a sequence of tokens (e.g., patches, rows, pixels). The RNN then processes these tokens sequentially to learn spatial relationships and extract image representations.

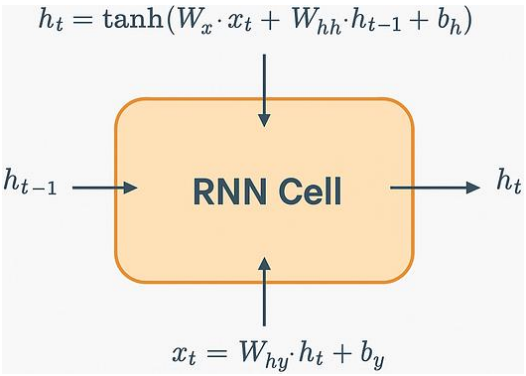


Figure 30. Recurrent Neural Networks basic cell

$$h_t = tanh(W_{xh} \cdot x_t + W_{hh} h_{t-1} + b_h)$$

$$y_t = W_{hy} \cdot h_t + b_y$$

Where:

- h_t : hidden state at time t
- h_{t-1} : hidden state at previous time
- x_t : input at time t
- W_{xh}, W_{hh}, W_{hy} : weight matrix
- b_h, b_y : biases

From formular, each RNN cell update its hidden state by combine the current input with information from previous time steps, allowing the network to learn sequential dependency.

5.2.1.1. Long Short Term Memory – LSTM

Observation of single LSTM:

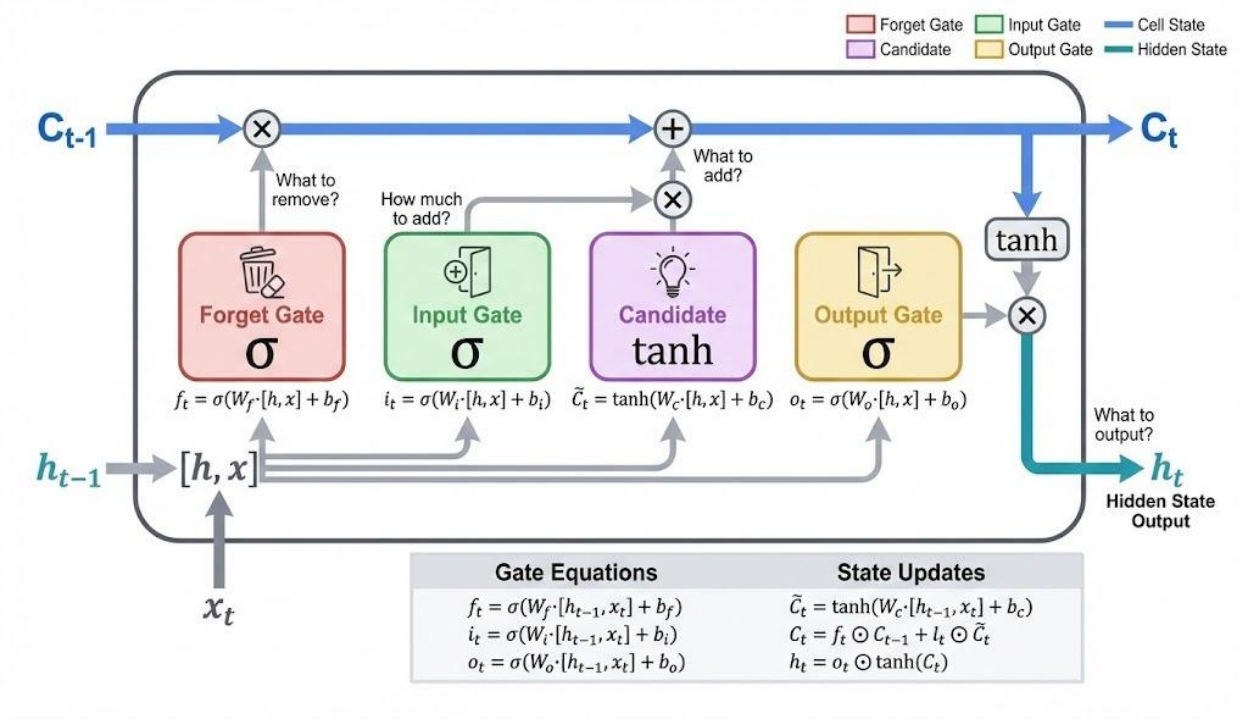


Figure 31. LSTM model consists of three gates and two states

Where:

- x_t : input at time t
- h_t : hidden state at time t
- h_{t-1} : hidden state at previous time

- C_t : update cell state, represent the new long-term memory
- C_{t-1} : previous cell state, represent long-term memory
- f_t : forget gate output – control how much previous cell state information is retained
- i_t : input gate output – control how much new candidate information is added
- o_t : output gate output – control how much cell state information is exposed as hidden state
- $\widetilde{C_t}$: candidate cell state – represent new candidate information to be stored in memory

There are three gates (forget gate, input gate, output gate) and two state (cell state and hidden state):

- Forget gate: this gate decides which information should be kept or removed from the cell state. It uses current input x_t and previous hidden state h_{t-1} then apply a sigmoid function to generate values between 0 and 1. Value close to 0 mean it refer to remove information, else close to 1 means refer retain information.
- Input gate: this gate determines which new information should be stored in the cell state. It uses current input and previous hidden state, then apply a sigmoid function to determine how much new information should be selected. A value close to 0 means less information is added, else close to 1 means more information is added. A candidate cell state is generated using tanh function to provide the new information.
- Output gate: this gate determines which information from the updated cell state should be exposed as the current hidden state. It uses current input and previous hidden state, then apply sigmoid function to control the output. The resulting hidden state is passed to the next time step and can be used for prediction.

5.2.1.2. Gated Recurrent Unit Networks (GRU)

Observation of single GRU:

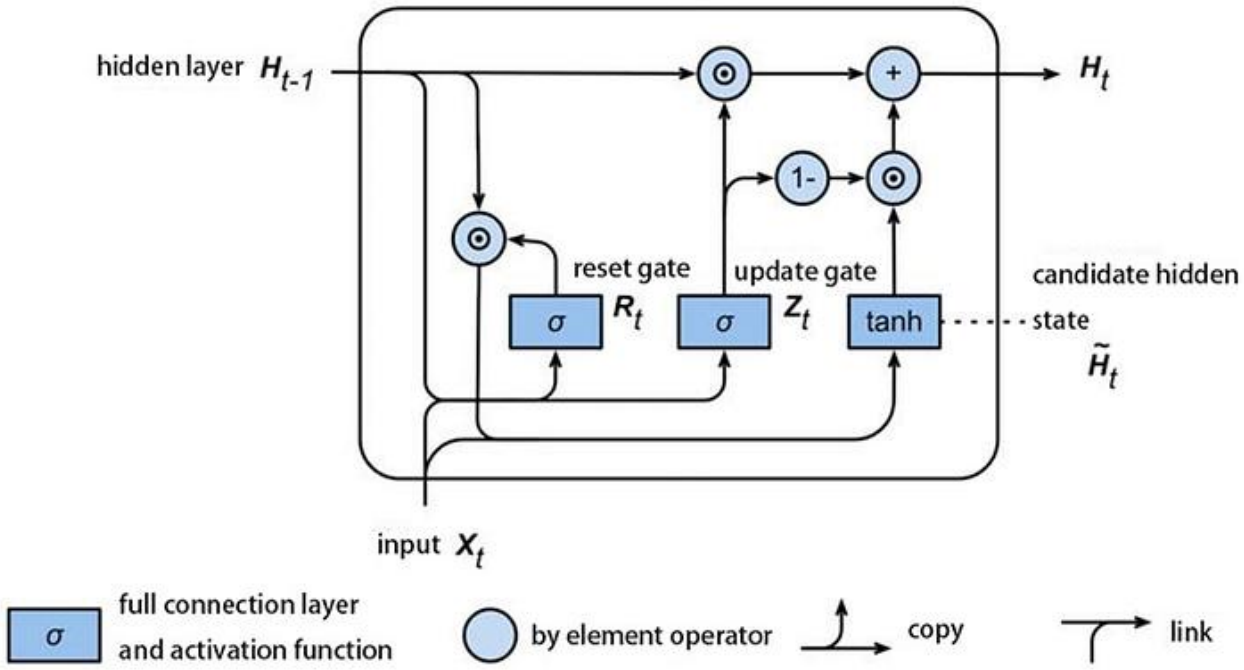


Figure 32. GRU model consists of two gates and one state

Where:

- x_t : input at time t
- h_t : hidden state at time t, combine previous information and new candidate information
- h_{t-1} : hidden state at previous time, carry information from the previous time step
- z_t : update gate – control how much new candidate information is incorporated compare to previous hidden state information retained
- r_t : reset gate – control how much previous hidden state information is used to generate the candidate hidden state
- $\tilde{h}$: candidate hidden state – represent new generated information that can be incorporated into the current hidden state

There are two gates (update gate, reset gate) and one state (hidden state):

- Reset gate: this gate determine how much of the previous hidden state is used to compute the new candidate state
- Update gate: this gate control how much of the new candidate state will be used to update the hidden state and how much of the previous hidden state is kept

5.2.2. Image classifier design for experiment

Design and implement a custom Recurrent neural Networks's image classifier to perform image classification on CIFAR-10. The architecture show as following:

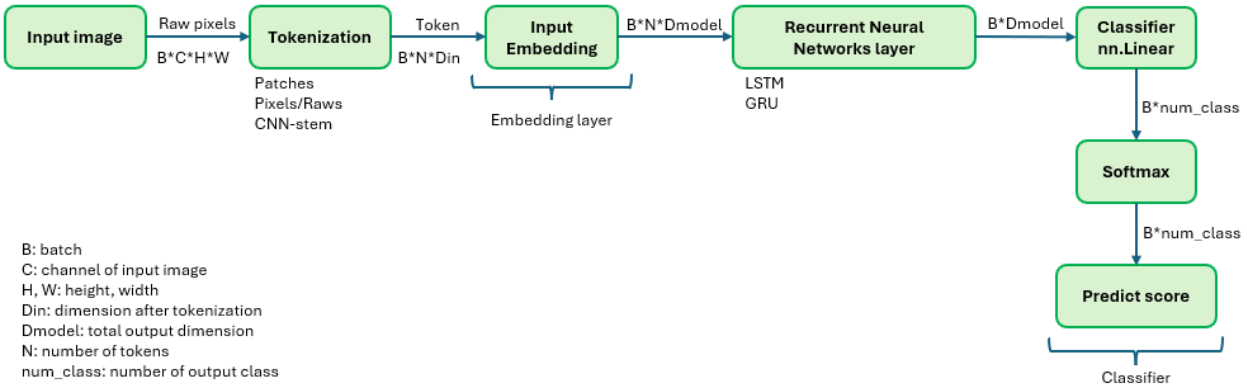


Figure 33. Full image classifier using RNNs block

- Input image: raw input of image (not a token)
- Tokenization: convert input images into sequence of token using different tokenization methods will be introduced (patch, pixel/row, CNNs extract feature map)
- Embedding layer: transforms each input token into model dimension before feeding it to RNNs
- RNN blocks process sequential inputs by maintaining and updating hidden states to capture dependencies and retain information across time steps
- Classifier and softmax: map aggregate value to predict scores for prediction

Re-use chapter 2 of general setup for training and evaluation pipeline. Review the following hyparameter should be apply:

batch_size	32
lr	0.01
mm	0.9
weight_decay	0.0005
num_epochs	50
step_size	10
gamma	0.1
num_worker	8
Loss function	Cross Entropy
Optimizer	SGD
Scheduler	StepLR

5.3. Tokenization Strategies

Implementation detailed will reuse from E2 (refer Chapter 4.4), following tokenization strategies will apply to this experiment:

- Patches tokenization (4*4)
- Row tokenization
- CNNs feature extraction tokenization

5.4. Implementation sequential model

5.4.1. Long Short-Term Memory Networks (LSTM)

Create class of image classifier model using LSTM:

```
class LSTMImageClassifier(nn.Module):
    def __init__(self, tokenizer, embedding_layer, embedding_dim,
hidden_dim, num_layers, num_classes, bidirectional=False):        (1)
        super().__init__()

        self.tokenizer = tokenizer                                (2)
        self.embedding_layer = embedding_layer
        self.bidirectional = bidirectional
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers,
batch_first=True, bidirectional=bidirectional)                    (3)
        self.classifier = nn.Linear(hidden_dim * (2 if bidirectional
else 1), num_classes)                                             (4)

    def forward(self, images_input):
        # Tokenization
        x = self.tokenizer(images_input)                          (2)

        # Embedding x using the embedding layer
        x = self.embedding_layer(x)

        # LSTM
        output, (h_n, c_n) = self.lstm(x)                         (3)

        # Last layer's final hidden state
        features = h_n[-1] if not self.bidirectional else
torch.cat((h_n[-2], h_n[-1]), dim=1)                              (5)

        # Classification (basic classification layer)
        logits = self.classifier(features)                        (4)

        return logits
```

(1) disabled birectional for both recurrent networks
(2) apply different tokenization strateries
(3) LSTM
(4) classifier layer
(5) return last layer final hidden state

5.4.2. Gated Recurrent Unit Networks (GRU)

Create class of image classifier model using GRU:

```
class GRUImageClassifier(nn.Module):
    def __init__(self, tokenizer, embedding_layer, embedding_dim,
hidden_dim, num_layers, num_classes, bidirectional=False):        (1)
        super().__init__()

        self.tokenizer = tokenizer                                (2)
```

(1) disabled birectional for both
(2) apply different tokenization strateries
(3) GRU

```
        self.embedding_layer = embedding_layer
        self.bidirectional = bidirectional
        self.gru = nn.GRU(embedding_dim, hidden_dim, num_layers,
batch_first=True, bidirectional=bidirectional)                    (3)
        self.classifier = nn.Linear(hidden_dim * (2 if bidirectional
else 1), num_classes)                                             (4)

    def forward(self, images_input):
        # Tokenization
        x = self.tokenizer(images_input)                          (2)

        # Embedding x using the embedding layer
        x = self.embedding_layer(x)

        # GRU
        output, h_n = self.gru(x)                                 (3)

        # Last layer's final hidden state
        features = h_n[-1] if not self.bidirectional else
torch.cat((h_n[-2], h_n[-1]), dim=1)                              (5)

        # Classification
        logits = self.classifier(features)                        (4)

        return logits
```

(4) classifier layer
(5) return last layer final hidden state

5.5. Experimental result and analysis

First of all, following table show the overall results of all models:

	LSTM			GRU		
Metrics	Patch	Row	CNNstem	Patch	Row	CNNstem
Train loss	1.18473	0.9476524	0.01716	0.868	0.74788	0.003075
Validation loss	1.2675687	1.171	0.4883835	1.04917	1.04877	0.47243
Test loss	1.2668	1.16917	0.69	1.0327321	1.05284	0.55244
Train accuracy (%)	56.83	66.3925	99.71	68.965	73.7525	99.9925
Validate accuracy (%)	54.06	59.0	84.67	63.41	63.42	86.71
Test accuracy (%)	53.72	58.76	84.12	62.98	63.21	86.26
Number of tokens	64	32	64	64	32	64
Marco Precision	0.5325	0.585	0.84107	0.627	0.629	0.863
Marco Recall	0.5372	0.5876	0.8412	0.6298	0.6321	0.8626
Marco F1-score	0.533	0.58578	0.841	0.628	0.6301	0.8626
Trainable parameters	403850	409994	2089428	304778	310922	1990356
Training time (s)	2220.81	2212.8	2591.3	2134.82	2220.2	3104.486
Testing time (s)	17.857	18.13	18.128	17.526	17.44	18.7336
Epoch	50	50	50	50	50	50

5.5.1. Comparison of different tokenization strategies

Three tokenization strategies, including Patch-based, Row-based, and CNN-stem Tokenization, were evaluated to investigate their impact on sequential image classification using LSTM and GRU.

❖ LSTM results

- Patches tokenization: it achieves 53.72% test accuracy with 403850 trainable parameters. Its relatively low accuracy suggests that the recurrent model may not effectively capture spatial relationships from the flattened patch sequence.
- Rows tokenization: improves test accuracy to 58.76%, with 409994 trainable parameters. Compared with patches tokenization, row sequences provide a more continuous spatial representation, allowing the recurrent model to process image information along each row.
- CNN-stem tokenization: achieves the highest test accuracy of 84.12%, with 2089428 trainable parameters. Using CNNs model to extracts spatial features before recurrent processing provides more informative representations for subsequent sequential modeling.

❖ GRU results

- Patches tokenization: it achieves 62.98% test accuracy with 304778 trainable parameters.
- Rows tokenization: it achieves 63.21% test accuracy with 310922 trainable parameters, showing only a slight improvement over patches tokenization.
- CNN-stem tokenization: it achieves the highest test accuracy of 86.26%, with 1990356 trainable parameters. Similar to LSTM, CNN-stem provides a substantial improvement over the other tokenization strategies.

CNN-stem consistently achieves the best classification performance for both LSTM and GRU, improving test accuracy by a considerable margin compared with patches and rows tokenization. This suggests that extracting spatial features using CNNs before recurrent processing is beneficial for CIFAR-10 image classification. However, CNN-stem introduces substantially higher model complexity, with approximately 2 million trainable parameters, compared with approximately 300K–410K for the other strategies. Its training time is also higher, particularly for GRU (3104.486 seconds). Therefore, CNN-stem provides the best classification performance, while patches and rows tokenization offer more parameter-efficient alternatives.

5.5.2. Comparison LSTM vs GRU

❖ Model capacity and training time

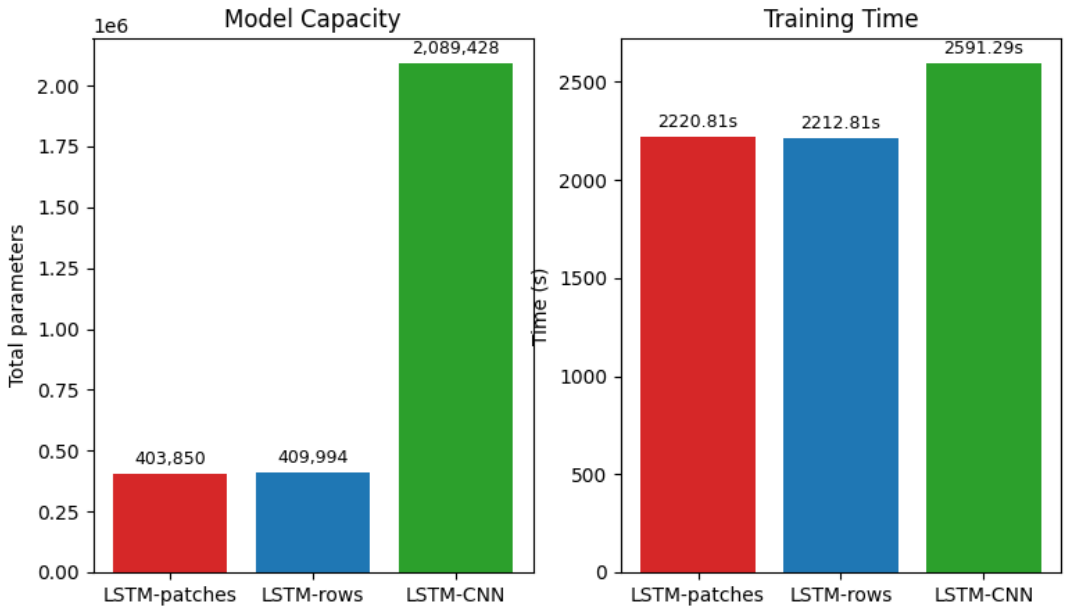


Figure 34. Model capacity and training time (model using LSTM)

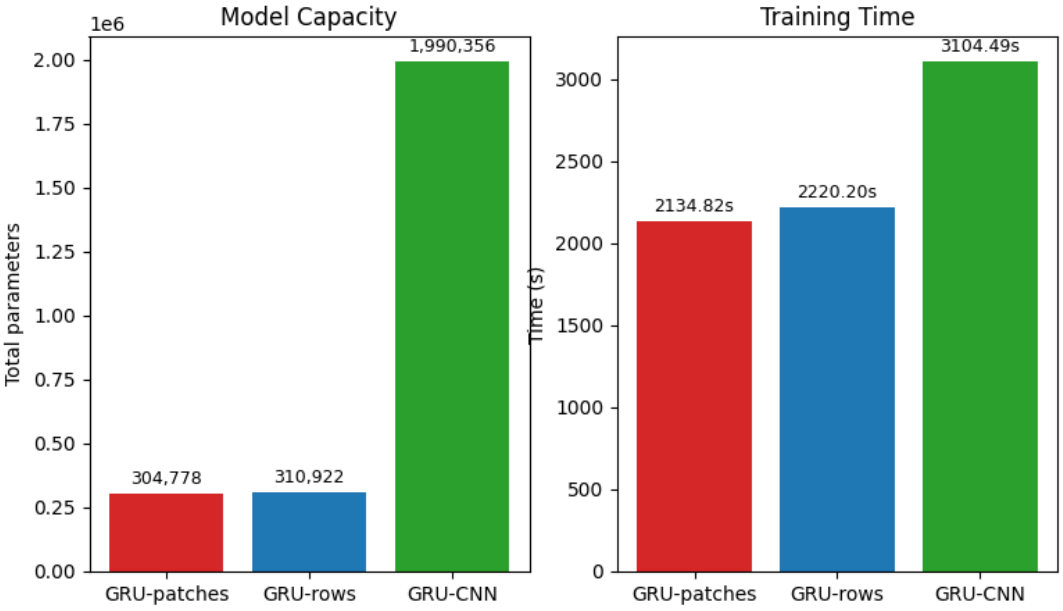


Figure 35. Model capacity and training time (model using GRU)

With model using LSTM or GRU sequential, tokenization take highest capacity that makes training time was higher than others. However, this capturing result indicate the trade-off between model capacity along with training time and its performance of classification.

GRU consistently has fewer trainable parameters than LSTM:

Tokenization	LSTM Parameters	GRU Parameters	Reduction
Patch	403850	304778	24.53%

Row	409994	310922	24.16%
CNN-stem	2089428	1990356	4.74%

This is consistent with the architectural differences between the two recurrent networks: GRU uses 2 gate (update gate, reset gate), while LSTM uses 3 gate (forget gate, input gate, output gate) and maintains separate hidden and cell states.

Tokenization	LSTM Training (s)	GRU Training (s)	LSTM Testing (s)	GRU Testing (s)
Patch	2220.81	2134.82	17.857	17.526
Row	2212.8	2220.2	18.13	17.44
CNN-stem	2591.30	3104.486	18.128	18.7336

GRU generally requires fewer parameters and achieves higher accuracy. However, its training time is not consistently lower. In particular, GRU with CNN-stem requires more training time than LSTM, even though it having fewer parameters. This indicates that parameter count alone does not determine actual computational efficiency.

❖ Classification Performance

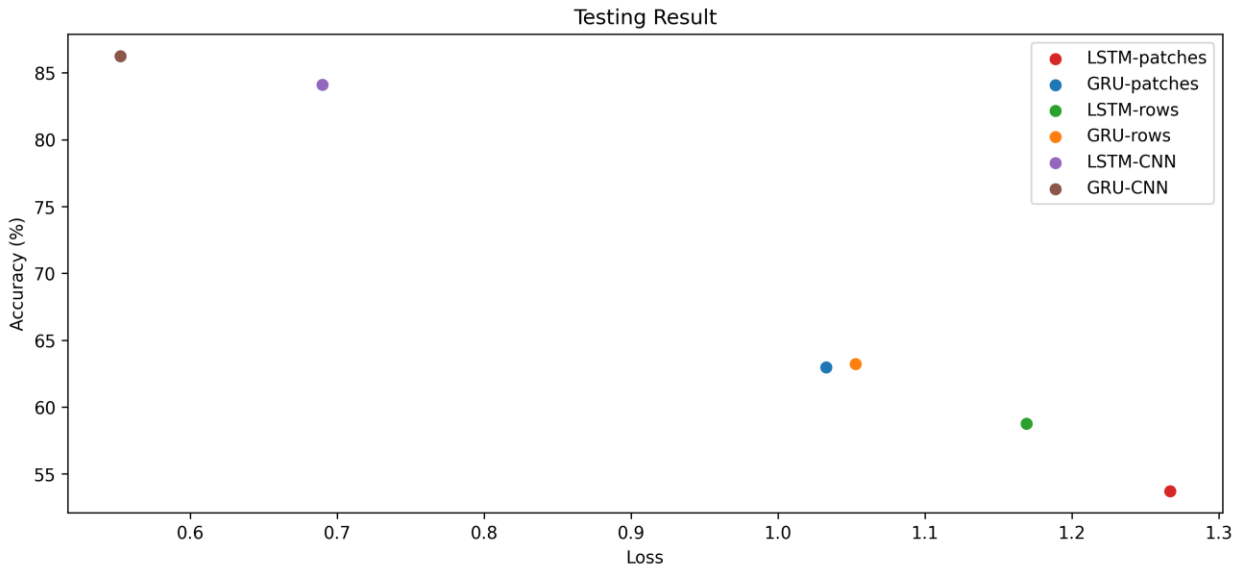


Figure 36. Testing result of all model (accuracy, loss)

GRU consistently outperforms LSTM across all 3 tokenization strategy:

Tokenization	LSTM Test Accuracy	GRU Test Accuracy	Improvement
Patch	53.72%	62.98%	+9.26 %
Row	58.76%	63.21%	+4.45 %
CNN-stem	84.12%	86.26%	+2.14 %

The largest improvement occurs with patches tokenization, with GRU achieves a 9.26 percentage increase over LSTM. With CNN-stem, the performance gap becomes smaller, suggesting that the CNN-extracted features provide a stronger representation for both recurrent architectures.

❖ Generalization Performance

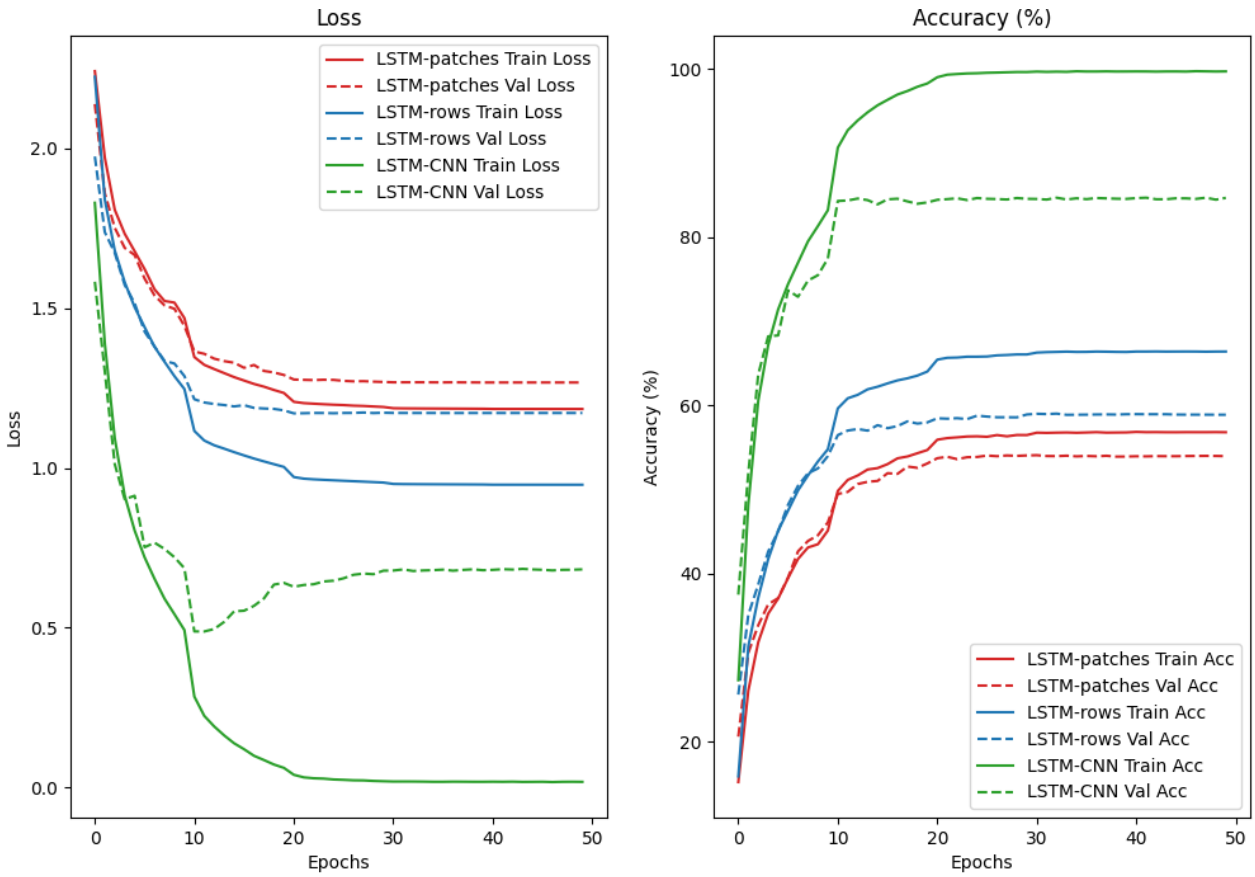


Figure 37. Loss/Accuracy curve of training/validation set (by LSTM)

The LSTM model training curves show that all 3 tokenization strategies progressively reduce training loss and improve training accuracy. However, their convergence behaviors and generalization performance differ considerably.

- Patches tokenization: model converges relatively slowly, with training accuracy reaching approximately 57% and validation accuracy stabilizing around 53.7%. Both loss curves gradually decrease before reaching a plateau with low accuracy and high loss, indicating limited learning capacity for this representation.
- Rows tokenization: it achieves better performance than patches tokenization, reaching approximately 66.4% training accuracy and 59% validation accuracy. The lower training and validation losses suggest that processing image rows as sequential

inputs provides more effective representations than the patches approach in this experiment.

- CNN-stem tokenization: it achieves the highest training and validation accuracy, reaching approximately 100% and 84.67%, respectively. Training loss nearly zeros, while validation loss initially decreases but subsequently increases and stabilizes at a higher value. The substantial gap between training and validation performance indicates significant overfitting. Although the CNN-stem extracts spatial features before recurrent processing, the model appears to memorize training samples rather than generalize equally well to unseen images.

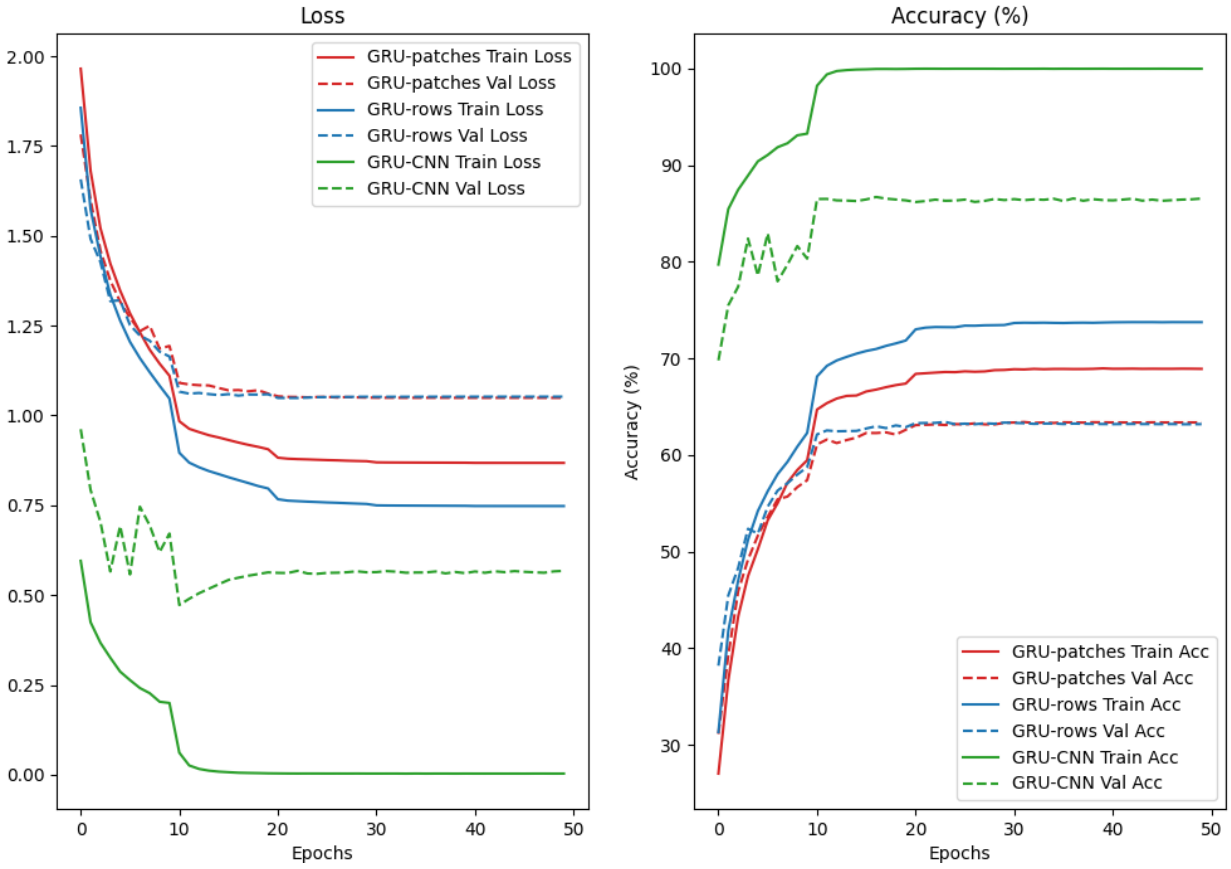


Figure 38. Loss/Accuracy curve of training/validation set (by GRU)

The GRU models exhibit similar convergence patterns but generally achieve better accuracy than when comparing with LSTM models.

- Patches tokenization: training accuracy reaches approximately 69%, while validation accuracy stabilizes around 63%. Compare with LSTM, GRU demonstrates a noticeable improvement in both training and validation performance.
- Rows tokenization: model achieves approximately 73.75% training accuracy and 63.42% validation accuracy. Although training performance improves over patches

tokenization, validation accuracy remains nearly unchanged, suggesting limited additional generalization benefits.

- CNN-stem tokenization: it achieves approximately 100% training accuracy and 86.71% validation accuracy. However, training loss approaches zero while validation loss remains higher. The unstable in validation loss during the early epochs, followed by a relatively stable plateau, also indicate overfitting.

❖ Overall

Comparing the two recurrent architectures, GRU consistently achieves higher validation accuracy than LSTM across all three tokenization strategies. The improvement is particularly noticeable with patch-based tokenization, while the difference becomes smaller when using CNN-stem features.

Regarding tokenization, CNN-stem consistently provides the strongest classification performance for both architectures. This suggests that extracting spatial features through convolutional layers before sequential processing is more effective for CIFAR-10 image classification than directly apply raw patches or image rows into recurrent layers. However, its substantially higher training accuracy compared with validation accuracy reveals a clear generalization gap.

In contrast, patches and rows models provide smaller differences between training and validation accuracy, but their overall classification performance is considerably lower. This indicates a trade-off between model fitting and generalization: CNN-stem achieves higher accuracy but is more susceptible to overfitting, while patches and rows representations show more limited learning performance.

Conclusion, GRU show better classification performance and lower model complexity across all 3 tokenization strategies. However, LSTM achieves shorter training time with CNN-stem. Both architectures benefit significantly from CNN-stem tokenization, indicating that the quality of input representation has a substantial impact on recurrent image classification. GRU is therefore the more parameter-efficient architecture in this experiments, while the choice between LSTM and GRU should also consider training time and the selected tokenization strategies.

5.5.3. Comparison with E1 baseline (MLP/CNN)

Review E1 baseline results on CNNs (convolution neural network to extract spatial feature on image), compare to CNN-stem tokenization of both LSTM and GRU:

Metrics	Baseline CNN	LSTM CNNstem	GRU CNNstem
Train loss	0.01432	0.01716	0.003075

Validation loss	0.333584	0.4883835	0.47243
Test loss	0.367557	0.69	0.55244
Train accuracy(%)	99.96	99.71	99.9925
Validate accuracy(%)	89.46	84.67	86.71
Test accuracy(%)	88.6	84.12	86.26
Marco Precision	0.88586	0.84107	0.863
Marco Recall	0.886	0.8412	0.8626
Marco F1-score	0.88579	0.841	0.8626
Trainable parameters	1667146	2089428	1990356
Training time (s)	3043.72	2591.3	3104.486

To investigate whether recurrent processing provides additional benefits after convolutional feature extraction, the E1 CNNs baseline is compared with the CNN-stem LSTM and CNN-stem GRU model:

- Classification Performance

The E1 CNNs achieves the highest test accuracy of 88.60%, followed by CNN-stem GRU at 86.26% and CNN-stem LSTM at 84.12%. The CNNs baseline also achieves the highest macro precision (0.8859), recall (0.8860), and F1-score (0.8858). These results indicate that the baseline CNNs provides the strongest classification performance among the 3 models. This suggests that adding recurrent processing to convolutional features does not necessarily improve classification performance for CIFAR-10.

- Generalization and Overfitting

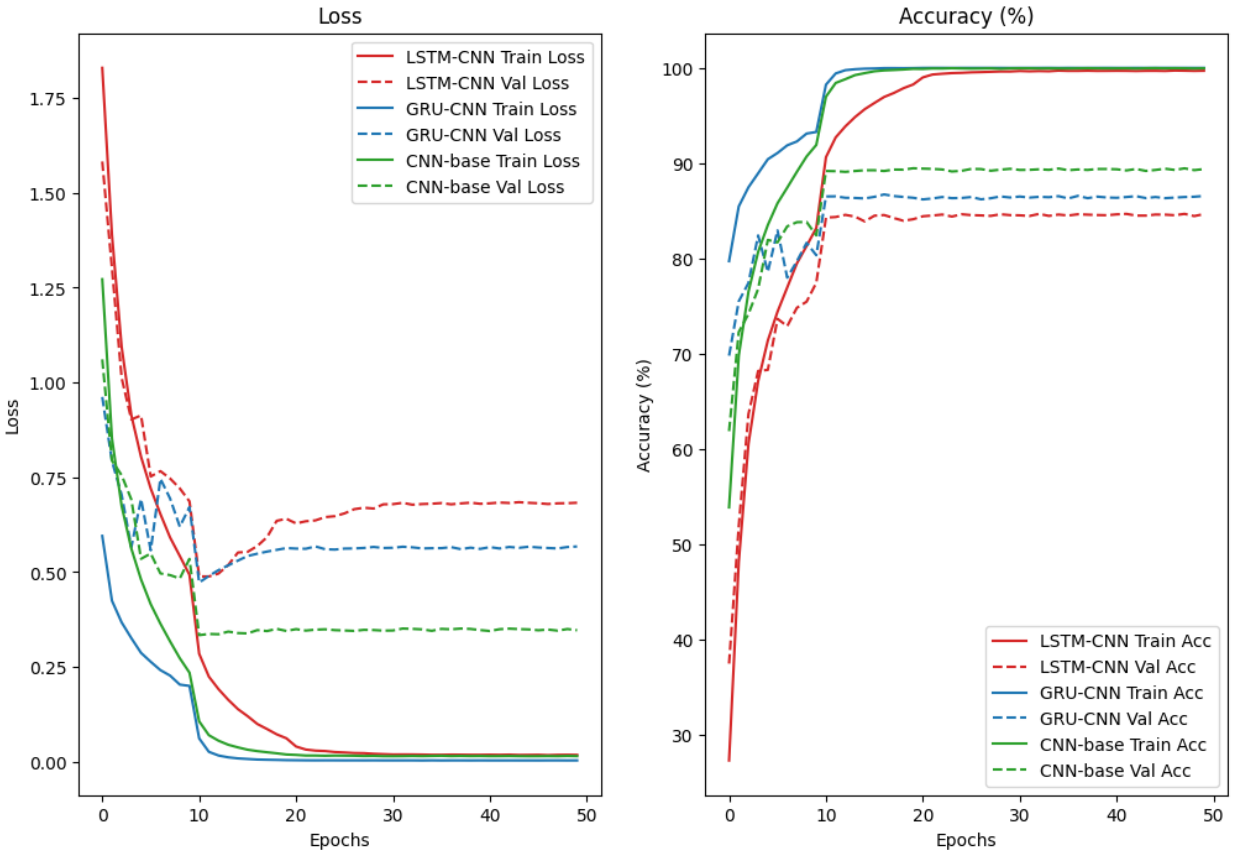


Figure 39. Loss/Accuracy curve of three models using CNN as base extract feature

All 3 models achieve very high training accuracy, exceeding 99%. However, their validation and test performance have many differences. The CNNs baseline has a training to test accuracy gap of approximately 11.36%, compared with 15.59% for CNN-stem LSTM and 13.73% for CNN-stem GRU. And, the CNNs baseline obtains the lowest validation loss (0.3336) and test loss (0.3676); both recurrent configurations have higher validation and test losses, indicating weaker generalization. CNN-stem LSTM exhibits the largest train–test performance gap, while CNN-stem GRU shows a smaller gap.

❖ Model Complexity and Training Efficiency

The CNNs baseline requires 1667146 parameters, while CNN-stem LSTM and CNN-stem GRU require 2089428 and 1990356 parameters. Therefore, combination of recurrent layers increases model complexity without improving test accuracy.

Regarding training time, CNN-stem LSTM is the fastest at 2591.3 seconds, followed by the CNNs baseline at 3043.72 seconds. CNN-stem GRU requires the longest training time at 3104.49 seconds. Although LSTM trains faster in this experiment, its classification performance is lower than others.

❖ Overall Discussion

The results demonstrate that the standalone CNNs provides the best balance between classification performance, generalization, and model complexity. While CNN-stem tokenization substantially improves the performance of recurrent models, the subsequent LSTM or GRU processing does not provide additional benefits over direct CNNs classification.

Between the 2 recurrent architectures, GRU achieves better classification accuracy with fewer trainable parameters than LSTM. However, its training time is slightly longer than that of the CNNs baseline. Overall, the findings suggest that convolutional feature extraction alone is sufficient for this CIFAR-10 classification task, and adding recurrent processing introduces computational complexity without a corresponding improvement in performance.

5.6. Discussion & key takeaways

The experimental results highlight the importance of image tokenization strategies and recurrent architecture selection for sequential image classification.

(1) CNN-stem - the most effective tokenization strategy for recurrent models:
   - CNN-stem consistently achieves the highest test accuracy among the evaluated recurrent configurations, reaching 84.12% with LSTM and 86.26% with GRU. This demonstrates that extracting spatial features through convolutional layers before sequential processing provides more effective representations than directly feeding raw image patches or rows into recurrent networks.

(2) GRU outperforms LSTM in classification performance and parameter efficiency:
   - Across all three tokenization strategies, GRU achieves higher test accuracy than LSTM while requiring fewer trainable parameters. The difference is particularly noticeable with patch-based tokenization, where GRU improves test accuracy by 9.26%. However, GRU does not consistently achieve shorter training times, indicating that fewer parameters do not necessarily translate into faster training.

(3) CNNs remains the strongest overall architecture:
   - Although CNN-stem improves recurrent model performance, neither LSTM nor GRU outperforms the E1 CNNs baseline. The standalone CNNs achieves 88.6% test accuracy with 1.67 million trainable parameters, outperforming CNN-stem GRU (86.26%) and CNN-stem LSTM (84.12%). This suggests that additional sequential processing does not provide sufficient benefits to justify its increased model complexity in this experiment.

(4) CNN-stem recurrent models exhibit overfitting:

- Both CNN-stem configurations achieve nearly 100% training accuracy, but their test accuracies remain considerably lower. This indicates a generalization gap, suggesting that the models may be overfitting the training data. Further improvements could apply data augmentation, dropout to avoid overfitting.

Overall, the experiments demonstrate that convolutional feature extraction is highly effective for CIFAR-10 image classification. CNN-stem provides a strong representation for recurrent models, while GRU offers a better accuracy–parameter trade-off than LSTM. Nevertheless, the standalone CNNs achieves the best overall classification performance with lower model complexity, indicating that recurrent processing is not necessarily advantageous for this particular image classification task.

References

[1] What Is Cross-Entropy Loss Function? - GeeksforGeeks

[2] CrossEntropyLoss — PyTorch main documentation

[3] Stochastic Gradient Descent with Momentum is Algorithmically Stable

[4] Frontiers | Integrating and optimizing genomic, weather, and secondary trait data for multiclass classification

[5] Evaluation Metrics in Machine Learning - GeeksforGeeks

[6] F1 Score in Machine Learning - GeeksforGeeks

[7] Vision Transformers (ViT) Explained | Pinecone

[8] MLIR Part 7 - Transformers

[9] Regularization Techniques in Machine Learning - GeeksforGeeks

[10] Multi-Head Attention Mechanism - GeeksforGeeks

[11] Vision Transformers vs. Convolutional Neural Networks (CNNs) - GeeksforGeeks

[12] Limitations of Single Attention Head

[13] Intro to Transformers: The Encoder Block | Edlitera

[14] How LLMs See Images, Audio, and More

[15] RNN, LSTM, GRU in NLP: A Deep Dive into Sequence Modeling | by Amit Kharche | Medium

[16] Modèles LSTM : guide complet des réseaux à mémoire à long et court terme | DataCamp

[17] RNN vs LSTM vs GRU vs Transformers - GeeksforGeeks

[18] What is a GRU? Gated Recurrent Units Explained (Architecture & Math)